



TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG NỀN TẢNG SPECHUB – TRA CỨU, SO SÁNH VÀ NGHIÊN CỨU THIẾT BỊ THÔNG MINH

Đơn vị thực hiện:

Nhóm phát triển Spechub

HÀ NỘI – 2026



TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG NỀN TẢNG SPECHUB – TRA CỨU, SO SÁNH VÀ NGHIÊN CỨU THIẾT BỊ THÔNG MINH

Đơn vị thực hiện:

Nhóm phát triển Spechub

Loại tài liệu:

Báo cáo đồ án tốt nghiệp

HÀ NỘI – 2026

TÓM TẮT

Spechub là nền tảng dữ liệu và nghiên cứu thiết bị thông minh, được xây dựng nhằm giải quyết tình trạng thông số phân tán, cách đặt tên không thống nhất và thiếu khả năng truy nguyên nguồn khi người dùng tìm hiểu sản phẩm. Hệ thống hợp nhất danh mục thiết bị, biến thể thương mại, linh kiện phân cứng, điểm đánh giá, lịch sử giá và nội dung Wiki trong một data model quan hệ có cấu trúc.

Giải pháp sử dụng kiến trúc monorepo với giao diện Next.js 15 và React 19, API NestJS 11 chạy trên Fastify 5, data access layer Prisma 6, PostgreSQL 16 cùng các mở rộng pgvector, pg_trgm và unaccent. Redis được dùng cho session, cache và điều phối tác vụ; Meilisearch là lựa chọn tăng cường cho tìm kiếm. Các chức năng chính gồm tra cứu và lọc thiết bị, so sánh, khuyến nghị theo nhu cầu, trợ lý AI dựa trên Retrieval-Augmented Generation (RAG) có citation, Wiki cộng tác, wishlist, cảnh báo giá, thông báo, quản trị catalog và API B2B.

Báo cáo trình bày quá trình khảo sát yêu cầu, lựa chọn kiến trúc, phân tích use case, thiết kế các luồng tương tác, xây dựng quy trình thu thập–duyệt dữ liệu và thiết kế database theo bốn miền. Kết quả khảo sát mã nguồn ghi nhận 22 web route, 30 controller API với 186 điểm cuối và 139 Prisma model. Trong lần đánh giá ngày 05/08/2026, 36 bộ test API gồm 189 ca test đều đạt; schema Prisma hợp lệ; PostgreSQL và Redis đáp ứng readiness check.

Kết quả cho thấy Spechub đã hình thành được nền tảng kỹ thuật có khả năng mở rộng theo miền nghiệp vụ, duy trì nguồn gốc dữ liệu và hỗ trợ nhiều nhóm người dùng. Các hướng phát triển quan trọng tiếp theo là hoàn thiện đo tải, end-to-end UI test (E2E), tăng độ phủ dữ liệu thực tế, xây dựng cơ chế đánh giá chất lượng câu trả lời AI và triển khai giám sát sản xuất theo mục tiêu mức dịch vụ.

Từ khóa: Spechub; danh mục thiết bị; so sánh; PostgreSQL; RAG; Wiki; cảnh báo giá.

LỜI CAM ĐOAN

Nhóm thực hiện xin cam đoan báo cáo đồ án tốt nghiệp này là kết quả của quá trình nghiên cứu, phân tích, thiết kế, xây dựng và đánh giá dự án Spechub. Nội dung báo cáo được trình bày trung thực, thống nhất với mã nguồn, schema dữ liệu, cấu hình hệ thống và kết quả test tại thời điểm đánh giá.

Các kiến thức, tài liệu và công nghệ được tham khảo đều được ghi rõ trong phần Tài liệu tham khảo. Những số liệu, hình ảnh giao diện và sơ đồ do nhóm thực hiện xây dựng được sử dụng đúng mục đích học thuật. Nhóm thực hiện chịu trách nhiệm về tính chính xác và trung thực của nội dung báo cáo.

Hà Nội, tháng 8 năm 2026

Nhóm phát triển Spechub

LỜI CẢM ƠN

Nhóm thực hiện xin trân trọng cảm ơn Trường Đại học Phenikaa và Khoa Công nghệ Thông tin đã tạo môi trường học tập, nghiên cứu thuận lợi; đồng thời trang bị những kiến thức cần thiết về kỹ nghệ phần mềm, database, phát triển ứng dụng web và bảo đảm chất lượng phần mềm.

Nhóm cũng ghi nhận đóng góp của cộng đồng mã nguồn mở đã phát triển Next.js, React, NestJS, Fastify, Prisma, PostgreSQL, Redis, Meilisearch và các công cụ test được sử dụng trong dự án. Mọi góp ý về phạm vi, chất lượng dữ liệu, an toàn hệ thống và trải nghiệm người dùng sẽ là cơ sở quan trọng để Spechub tiếp tục hoàn thiện.

MỤC LỤC

TÓM TẮT.....	i
LỜI CAM ĐOAN.....	ii
LỜI CẢM ƠN.....	iii
DANH MỤC HÌNH ẢNH.....	vi
DANH MỤC BẢNG BIỂU.....	viii
DANH MỤC TỪ VIẾT TẮT.....	x
MỞ ĐẦU.....	1
1. Lý do chọn đề tài.....	1
2. Mục tiêu.....	1
3. Đối tượng và phạm vi.....	2
4. Phương pháp thực hiện.....	2
5. Đóng góp của đồ án.....	2
6. Cấu trúc báo cáo.....	2
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI.....	3
1.1. Bối cảnh và bài toán.....	3
1.2. Khoảng trống của các cách tiếp cận hiện có.....	4
1.3. Tầm nhìn và phạm vi sản phẩm.....	4
1.4. Các bên liên quan.....	5
1.5. Yêu cầu chức năng.....	6
1.6. Yêu cầu phi chức năng.....	7
1.7. Rủi ro và ràng buộc.....	8
1.8. Kết luận chương.....	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	9
2.1. Nguyên tắc kiến trúc.....	9
2.2. Monorepo và tổ chức mã nguồn.....	10
2.3. Presentation layer với Next.js và React.....	11
2.4. API với NestJS và Fastify.....	11
2.5. Prisma, PostgreSQL và tìm kiếm dữ liệu.....	12

MỤC LỤC (TIẾP)

2.6. Redis, background job và resilience.....	12
2.7. Retrieval-Augmented Generation (RAG) và AI có citation.....	13
2.8. Authentication, authorization và security.....	13
2.9. PWA, observability và triển khai.....	14
2.10. Kết luận chương.....	16
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	18
3.1. Tác nhân và mô hình use case.....	18
3.2. Đặc tả use case.....	21
3.3. Thiết kế sequence diagram.....	29
3.4. Thiết kế background workflow.....	32
3.5. Thiết kế dữ liệu.....	34
3.6. Thiết kế API và data contract.....	39
3.7. Thiết kế an toàn và kiểm soát chất lượng.....	40
3.8. Kết luận chương.....	41
CHƯƠNG 4. PHÁT TRIỂN VÀ KẾT QUẢ.....	42
4.1. Môi trường và quy trình phát triển.....	42
4.2. Hiện thực các module chính.....	43
4.3. Kết quả giao diện.....	43
4.4. Kết quả API và dữ liệu.....	53
4.5. Test và đánh giá.....	54
4.6. Đánh giá mức độ đáp ứng.....	55
4.7. Hạn chế.....	56
4.8. Kết luận chương.....	57
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	58
TÀI LIỆU THAM KHẢO.....	59
PHỤ LỤC A. DANH MỤC API.....	60
PHỤ LỤC B. DANH MỤC MÔ HÌNH DỮ LIỆU.....	62
PHỤ LỤC C. HƯỚNG DẪN VẬN HÀNH VÀ KIỂM THỬ.....	67
PHỤ LỤC D. HƯỚNG DẪN SỬ DỤNG TÓM TẮT.....	69

DANH MỤC HÌNH ẢNH

Hình 1.1. Sơ đồ context hệ thống Spechub.....	3
Hình 2.1. Cấu trúc monorepo Spechub.....	10
Hình 2.2. Kiến trúc logic nhiều lớp.....	15
Hình 2.3. Mô hình triển khai tham chiếu.....	16
Hình 3.1. Use case phía người dùng.....	19
Hình 3.2. Use case biên tập và quản trị.....	20
Hình 3.3. Sequence diagram đăng nhập và refresh session.....	30
Hình 3.4. Sequence diagram tìm kiếm và so sánh.....	31
Hình 3.5. Sequence diagram trợ lý AI theo RAG.....	32
Hình 3.6. Workflow theo dõi giá và phát cảnh báo.....	33
Hình 3.7. Ingestion workflow.....	34
Hình 3.8. ERD nhóm catalog.....	36
Hình 3.9. ERD nhóm hardware và scoring.....	37

DANH MỤC HÌNH ẢNH (TIẾP)

Hình 3.10. ERD nhóm content, crawler và AI/search.....	38
Hình 3.11. ERD nhóm user engagement, subscription và B2B.....	39
Hình 4.1. Trang chủ Spechub.....	45
Hình 4.2. Trang danh mục thiết bị.....	46
Hình 4.3. Trang chi tiết thiết bị.....	47
Hình 4.4. Kết quả tìm kiếm thiết bị và phần cứng.....	48
Hình 4.5. Giao diện chọn thiết bị để so sánh.....	49
Hình 4.6. Quy trình khuyến nghị theo nhu cầu.....	50
Hình 4.7. Giao diện trợ lý AI.....	51
Hình 4.8. Giao diện Wiki công nghệ.....	52
Hình 4.9. Giao diện đăng nhập.....	53

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh cách tiếp cận dữ liệu thiết bị.....	4
Bảng 1.2. Các bên liên quan.....	5
Bảng 1.3. Yêu cầu chức năng chính.....	6
Bảng 1.4. Yêu cầu phi chức năng.....	7
Bảng 1.5. Rủi ro và biện pháp giảm thiểu.....	8
Bảng 2.1. Thành phần công nghệ và phiên bản.....	9
Bảng 2.2. Biện pháp security theo lớp.....	13
Bảng 2.3. Lựa chọn kiến trúc và đánh đổi.....	16
Bảng 3.1. Tác nhân hệ thống.....	18
Bảng 3.2. Danh mục use case.....	20
Bảng 3.3. Đặc tả UC-01 – Tìm kiếm và lọc thiết bị.....	22
Bảng 3.4. Đặc tả UC-02 – So sánh thiết bị.....	23
Bảng 3.5. Đặc tả UC-03 – Nhận khuyến nghị và hỏi AI.....	24
Bảng 3.6. Đặc tả UC-04 – Đăng nhập và quản lý phiên.....	25
Bảng 3.7. Đặc tả UC-05 – Wishlist và cảnh báo giá.....	26
Bảng 3.8. Đặc tả UC-06 – Đóng góp và duyệt Wiki.....	27

DANH MỤC BẢNG BIỂU (TIẾP)

Bảng 3.9. Đặc tả UC-07 – Thu thập và duyệt catalog.....	28
Bảng 3.10. Đặc tả UC-08 – Khai thác API B2B.....	29
Bảng 3.11. Phân nhóm 139 data model.....	34
Bảng 3.12. Quy ước hợp đồng API.....	39
Bảng 4.1. Môi trường phần mềm.....	42
Bảng 4.2. Module hiện thực chính.....	43
Bảng 4.3. Bản đồ route giao diện.....	44
Bảng 4.4. Thống kê endpoint API.....	53
Bảng 4.5. Bảng chứng test và kiểm chứng.....	55
Bảng 4.6. Đánh giá mức độ đáp ứng yêu cầu.....	55
Bảng 4.7. Hạn chế và tác động.....	56
Bảng A.1. Danh mục controller và endpoint.....	60
Bảng B.1. Danh sách Prisma model.....	62
Bảng C.1. Ma trận lệnh vận hành và mục đích.....	67
Bảng D.1. Tác vụ người dùng thường gặp.....	69

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Ý nghĩa
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
B2B	Business-to-Business	Dịch vụ giữa các tổ chức
CI	Continuous Integration	Tích hợp liên tục
CRUD	Create, Read, Update, Delete	Các thao tác dữ liệu cơ bản
CSR	Client-Side Rendering	Kết xuất phía trình duyệt
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
ERD	Entity–Relationship Diagram	Sơ đồ thực thể–quan hệ
FK	Foreign Key	Foreign key
HTTP(S)	Hypertext Transfer Protocol (Secure)	Giao thức truyền siêu văn bản (security)
JWT	JSON Web Token	Mã thông báo web JSON
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NFR	Non-functional Requirement	Yêu cầu phi chức năng
ORM	Object–Relational Mapping	Ánh xạ đối tượng–quan hệ
PK	Primary Key	Khóa chính
PWA	Progressive Web Application	Ứng dụng web tiến bộ
RAG	Retrieval-Augmented Generation	Sinh nội dung có Retrieval-Augmented Generation (RAG)
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò
REST	Representational State Transfer	Phong cách kiến trúc dịch vụ web
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SLO	Service Level Objective	Mục tiêu mức dịch vụ

Từ viết tắt	Tiếng Anh	Ý nghĩa
SPA	Single-Page Application	Ứng dụng đơn trang
SSR	Server-Side Rendering	Kết xuất server-side
UI/UX	User Interface / User Experience	Giao diện / trải nghiệm người dùng
UUID	Universally Unique Identifier	Định danh duy nhất phổ quát

MỞ ĐẦU

1. Lý do chọn đề tài

Thị trường thiết bị thông minh phát triển nhanh về số lượng model, biến thể bộ nhớ, cấu hình phần cứng và phiên bản phần mềm. Cùng một sản phẩm có thể được mô tả bằng nhiều tên thương mại, đơn vị đo hoặc tập thông số khác nhau giữa trang hãng, cửa hàng và bài đánh giá. Người dùng phải mở nhiều nguồn, tự đối chiếu và thường không biết một con số bắt nguồn từ đâu.

Các trang thương mại điện tử tối ưu cho giao dịch, trong khi các bảng thông số đơn lẻ thường thiếu quan hệ giữa thiết bị, chipset, CPU, GPU, màn hình, camera và bằng chứng nguồn. Khi bổ sung AI, nếu hệ thống không ràng buộc câu trả lời vào catalog đã chuẩn hóa và citation, nguy cơ đưa ra thông tin thiếu căn cứ càng tăng. Spechub được lựa chọn để nghiên cứu một nền tảng coi dữ liệu, nguồn gốc và quy trình duyệt là lõi, thay vì chỉ là một giao diện tìm kiếm.

2. Mục tiêu

Mục tiêu tổng quát là xây dựng nền tảng web phục vụ tra cứu, so sánh và nghiên cứu thiết bị thông minh trên một data model có cấu trúc, có thể truy nguyên nguồn và mở rộng cho tác vụ AI. Các mục tiêu cụ thể gồm:

- Chuẩn hóa danh mục model, variant, linh kiện và thuộc tính kỹ thuật.
- Cung cấp tìm kiếm, lọc, xem chi tiết, so sánh và khuyến nghị theo nhu cầu.
- Thiết kế trợ lý AI/RAG sử dụng context catalog và trả về citation.
- Tổ chức Wiki có phiên bản, kiểm duyệt, bình luận và liên kết nguồn.
- Hỗ trợ wishlist, lịch sử giá, cảnh báo và thông báo theo kênh.
- Cung cấp công cụ biên tập, audit log, kiểm tra trạng thái, metrics và API dành cho đối tác có hạn mức.
- Đánh giá hiện thực bằng test tự động, kiểm tra schema và readiness hạ tầng.

3. Đối tượng và phạm vi

Đối tượng nghiên cứu gồm kiến trúc ứng dụng web nhiều lớp, thiết kế database danh mục, tìm kiếm toàn văn và vector, quản lý phiên và authorization, quy trình biên tập dữ liệu, RAG có citation, background job và test API. Phạm vi thực hiện bao gồm giao diện web, API, worker, các shared package và hạ tầng PostgreSQL/Redis. Việc đánh giá tải lớn trên môi trường Internet và mức độ bao phủ toàn bộ thiết bị trên thị trường không nằm trong phạm vi của đề án.

4. Phương pháp thực hiện

1. Khảo sát bài toán, xác định đối tượng sử dụng, phạm vi nghiên cứu và các yêu cầu của hệ thống.
2. Phân tích tài liệu dự án, cấu trúc mã nguồn, các controller API, web route và schema Prisma.
3. Vận hành hệ thống trong môi trường cục bộ, readiness check, ghi nhận giao diện và thực hiện các bộ test API.
4. Mô hình hóa tác nhân, use case, biểu đồ tuần tự, quy trình nghiệp vụ và ERD theo từng miền dữ liệu.
5. Đối chiếu kết quả xây dựng với yêu cầu, phân tích các hạn chế và đề xuất hướng phát triển tiếp theo.

5. Đóng góp của đề án

Đề án đóng góp một data model sâu cho thiết bị thông minh; một kiến trúc monorepo tách ứng dụng triển khai và shared package; một bề mặt API có authorization; luồng AI gắn với dữ liệu và citation; cùng quy trình biên tập không cho dữ liệu ngoài đi thẳng vào catalog công khai. Báo cáo cũng đưa ra bộ sơ đồ và phụ lục định lượng để các thành viên mới có thể hiểu nhanh hệ thống.

6. Cấu trúc báo cáo

Sau phần mở đầu, Chương 1 trình bày bối cảnh, yêu cầu và ràng buộc; Chương 2 giải thích cơ sở lý thuyết và các lựa chọn công nghệ; Chương 3 đặc tả use case, luồng tương tác và dữ liệu; Chương 4 mô tả hiện thực, giao diện và kết quả test. Phần cuối tổng kết, nêu hướng phát triển, tài liệu tham khảo và các phụ lục API, data model, vận hành và hướng dẫn sử dụng.

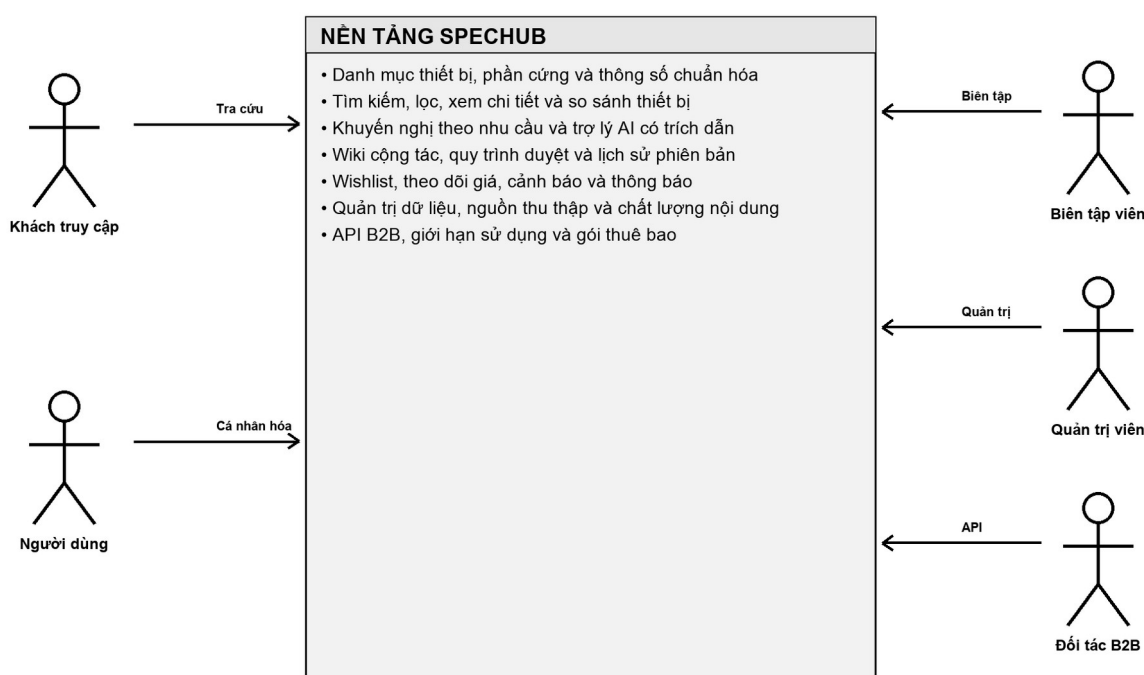
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Bối cảnh và bài toán

Một quyết định mua thiết bị không chỉ phụ thuộc giá niêm yết. Người dùng cần biết khác biệt giữa các variant, cấu hình chipset, chất lượng màn hình, camera, pin, khả năng cập nhật hệ điều hành và mức phù hợp với nhu cầu. Dữ liệu này thường nằm rải rác ở trang hãng, cửa hàng, cơ sở benchmark và bài viết kỹ thuật. Tên trường, đơn vị và mức chi tiết khác nhau khiến việc tổng hợp thủ công tốn thời gian và dễ sai.

Ở góc độ kỹ thuật, dữ liệu thiết bị có tính quan hệ cao. Một model có nhiều variant; một variant dùng nhiều thành phần; một linh kiện có thể xuất hiện trên nhiều thiết bị; một giá trị có thể cần nhiều nguồn xác nhận và thay đổi theo thời gian. Nếu lưu toàn bộ dưới dạng văn bản hoặc JSON không được quản trị, việc tìm kiếm, so sánh, chấm điểm và duy trì tính nhất quán sẽ khó mở rộng.

Spechub xác định ba bài toán trung tâm: chuẩn hóa tri thức thiết bị; cung cấp trải nghiệm khám phá có thể giải thích; và duy trì vòng đời dữ liệu từ thu thập, đối soát, xuất bản đến indexing. Các chức năng thương mại như wishlist, cảnh báo giá, affiliate và gói API được thiết kế xoay quanh lõi dữ liệu đó.



Hình 1.1. Sơ đồ context hệ thống Spechub

1.2. Khoảng trống của các cách tiếp cận hiện có

Không có một kiểu sản phẩm đơn lẻ đáp ứng đồng thời chiều sâu dữ liệu, trải nghiệm so sánh, cộng tác nội dung, khả năng truy nguyên và quyền truy cập lập trình. Bảng 1.1 tổng hợp sự khác biệt theo mục tiêu sử dụng; đây là phân tích định tính về loại hình giải pháp, không phải xếp hạng một nhà cung cấp cụ thể.

Bảng 1.1. So sánh cách tiếp cận dữ liệu thiết bị

Cách tiếp cận	Điểm mạnh	Khoảng trống đối với Spechub
Trang hãng	Thông tin chính thức, hình ảnh chuẩn	Khó so sánh chéo; dữ liệu thường chỉ tập trung sản phẩm của hãng
Sàn/cửa hàng	Giá và tồn kho gần giao dịch	Thông số có thể rút gọn, lặp hoặc không đồng nhất giữa người bán
Trang thông số	Tra cứu nhanh nhiều model	Quan hệ linh kiện, nguồn citation và quy trình duyệt thường hạn chế
Bài đánh giá	Phân tích sâu, có context sử dụng	Khó cấu trúc hóa để lọc, so sánh tự động và cập nhật hàng loạt
Chatbot tổng quát	Hội thoại tự nhiên	Có nguy cơ trả lời không bám catalog, thiếu citation và không tái lập
Spechub	Kết hợp catalog, nguồn, Wiki, so sánh, RAG và cảnh báo	Đòi hỏi đầu tư lớn cho chuẩn hóa, kiểm duyệt và vận hành dữ liệu

Khoảng trống mà Spechub ưu tiên không phải là số lượng trang, mà là khả năng liên kết dữ liệu có cấu trúc với bằng chứng. Khi một thuộc tính xuất hiện trong kết quả so sánh hoặc câu trả lời AI, hệ thống cần xác định được thực thể liên quan, phiên bản dữ liệu và nguồn tham chiếu phù hợp.

1.3. Tầm nhìn và phạm vi sản phẩm

Tầm nhìn của Spechub là trở thành lớp tri thức dùng chung cho nghiên cứu thiết bị thông minh. Lớp trải nghiệm phục vụ người dùng cuối; Catalog Studio phục vụ quản trị dữ liệu; API B2B mở quyền truy cập có kiểm soát; còn worker duy trì các tác vụ dài và

định kỳ. Phạm vi mã nguồn quan sát được bao gồm 22 route web, 30 controller API, 186 endpoint và 139 data model.

- Catalog: loại thiết bị, hãng/tổ chức, họ sản phẩm, model, variant, alias, media và trạng thái phát hành.
- Phần cứng: chipset, CPU, GPU, NPU, màn hình, camera, pin, bộ nhớ, lưu trữ, hệ điều hành và benchmark.
- Khám phá: tìm kiếm, lọc, xem chi tiết, so sánh, chấm điểm và khuyến nghị.
- Nội dung: nguồn, citation, Wiki, phiên bản, bình luận, kiểm duyệt và embedding.
- Cá nhân hóa: tài khoản, wishlist, cảnh báo giá, thông báo và tùy chọn kênh nhận.
- Thương mại và vận hành: tiếp thị liên kết, lịch sử giá, gói thuê bao, Stripe webhook, API key, thống kê sử dụng, audit log, kiểm tra trạng thái và metrics.

1.4. Các bên liên quan

Bảng 1.2. Các bên liên quan

Bên liên quan	Nhu cầu chính	Tiêu chí thành công
Khách truy cập	Tìm và so sánh nhanh mà không buộc đăng nhập	Kết quả rõ, có bộ lọc, có nguồn và URL chia sẻ được
Người dùng	Lưu danh sách, theo dõi giá, nhận thông báo	Dữ liệu cá nhân tách biệt, cảnh báo đúng điều kiện, không gửi lặp
Biên tập viên	Cập nhật catalog và nội dung từ nguồn	Có queue duyệt, diff, citation và audit
Kiểm duyệt viên	Đánh giá thay đổi và phiên bản Wiki	Phê duyệt/từ chối có lý do, authorization đúng
Quản trị viên	Quản lý quyền, gói dịch vụ và vận hành	Có dashboard, health, metrics, log và cơ chế thu hồi
Đối tác B2B	Truy cập dữ liệu ổn định bằng API	API key có scope, hạn mức, thống kê usage và tài liệu hợp đồng
Nhóm phát triển	Phát triển nhiều module trong một kho	Build/test nhất quán, kiểu dữ liệu dùng chung, lỗi dễ truy vết

1.5. Yêu cầu chức năng

Yêu cầu chức năng được nhóm theo hành trình người dùng và vòng đời dữ liệu. Mỗi yêu cầu có mã để đối chiếu ở Chương 4.

Bảng 1.3. Yêu cầu chức năng chính

Mã	Yêu cầu	Mức ưu tiên	Bảng chứng hiện thực
FR-01	Tìm kiếm thiết bị/phần cứng theo từ khóa và bộ lọc	Bắt buộc	Route /search, controller search/catalog
FR-02	Xem model, variant, thông số, điểm và nguồn	Bắt buộc	Route /devices/[slug], API catalog
FR-03	So sánh nhiều thiết bị trên thuộc tính chuẩn hóa	Bắt buộc	Route /compare, compare service
FR-04	Đề xuất thiết bị theo nhu cầu	Cao	Route /recommend, AI recommendations
FR-05	Hỏi AI và nhận câu trả lời có citation	Cao	Route /ai, /ai/ask và /ai/chat
FR-06	Đăng ký, đăng nhập, làm mới và thu hồi phiên	Bắt buộc	Auth API, JWT và Redis session
FR-07	Quản lý wishlist và cảnh báo giá	Cao	Route /wishlist, /alerts và worker
FR-08	Đọc, soạn, sửa và duyệt Wiki theo phiên bản	Cao	Các route /wiki và API moderation
FR-09	Thu thập, chuẩn hóa và duyệt dữ liệu danh mục	Bắt buộc	Nguồn dữ liệu, raw page và quy trình kiểm duyệt
FR-10	Quản trị quyền, gói thuê bao, audit log và webhook	Cao	Các controller quản trị, thanh toán và audit log
FR-11	Cấp API key, phạm vi quyền và theo dõi mức sử dụng B2B	Trung bình	Tuyến /api-access và các controller B2B
FR-12	Cung cấp kiểm tra trạng thái, readiness và metrics	Bắt buộc	Bốn điểm cuối kiểm tra trạng thái hoạt động

1.6. Yêu cầu phi chức năng

Bảng 1.4. Yêu cầu phi chức năng

Mã	Thuộc tính	Yêu cầu có thể kiểm chứng
NFR-01	An toàn	Validate DTO theo whitelist; từ chối trường lạ; JWT và RBAC; Helmet; CORS giới hạn; không lưu API key dạng rõ.
NFR-02	Tính đúng	Ràng buộc khóa/unique trong DB; quy trình duyệt; citation; test service/controller; schema Prisma hợp lệ.
NFR-03	Khả dụng	Health/liveness/readiness tách biệt; phụ thuộc DB/Redis được báo rõ; lỗi có request ID.
NFR-04	Hiệu năng	Phân trang, index, cache Redis, tìm kiếm chuyên dụng tùy chọn; tác vụ dài chạy nền.
NFR-05	Mở rộng	Module NestJS theo miền; monorepo; dữ liệu chuẩn hóa; API version /v1.
NFR-06	Quan sát	Structured log, request ID, metrics và audit cho hành động đặc quyền.
NFR-07	Trải nghiệm	Giao diện responsive, PWA/offline, thông báo trạng thái và thao tác chính có thể dùng bàn phím.
NFR-08	Bảo trì	TypeScript strict, schema dùng chung, lint/test, tài liệu Swagger trong môi trường phát triển.

Một số mục như ngưỡng latency theo phân vị, tải đồng thời và mục tiêu mức dịch vụ trong môi trường vận hành chưa có kết quả đo tải trong phạm vi đề án. Vì vậy, các nội dung này được trình bày dưới dạng mục tiêu thiết kế và hướng đánh giá tiếp theo, không sử dụng số liệu hiệu năng giả định.

1.7. Rủi ro và ràng buộc

Bảng 1.5. Rủi ro và biện pháp giảm thiểu

Rủi ro	Khả năng / tác động	Biện pháp giảm thiểu
Nguồn ngoài thay đổi cấu trúc	Cao / Cao	Lưu raw page, content hash, parser theo nguồn, queue review và cảnh báo thất bại
Thông số xung đột	Cao / Cao	Chuẩn hóa đơn vị, lưu citation, trust score và yêu cầu đối soát trước xuất bản
AI tạo nội dung thiếu căn cứ	Trung bình / Cao	RAG top-k, prompt policy, citation ID, cache có phiên bản và đánh giá thủ công
Lộ refresh token/API key	Thấp / Cao	Cookie security, hash key, session Redis, scope, rate limit và thu hồi
Tăng trưởng schema	Cao / Trung bình	Chia miền, join table, migration, quy ước đặt tên và phụ lục mô hình
Phụ thuộc dịch vụ ngoài	Trung bình / Trung bình	Timeout, fallback, retry có giới hạn, webhook idempotency và degrade có kiểm soát
Thiếu dữ liệu bao phủ	Cao / Trung bình	Ưu tiên chất lượng hơn số lượng, đo độ đầy đủ theo category và mở rộng theo lộ trình

1.8. Kết luận chương

Chương 1 đã xác định Spechub là bài toán quản trị tri thức thiết bị chứ không chỉ là một website liệt kê sản phẩm. Yêu cầu trọng tâm là cấu trúc dữ liệu sâu, truy nguyên nguồn, khám phá dễ dàng, kiểm soát thay đổi và bề mặt API an toàn. Các yêu cầu và rủi ro này là cơ sở để lựa chọn công nghệ ở Chương 2 và đặc tả thiết kế ở Chương 3.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Nguyên tắc kiến trúc

Kiến trúc Spechub tuân theo các nguyên tắc: phân tách trách nhiệm; data contract rõ; authentication ở biên và authorization ở nghiệp vụ; mọi thay đổi quan trọng có audit; dữ liệu ngoài phải qua bước chuẩn hóa/duyet; tác vụ dài được tách khỏi request; và chức năng AI phải có khả năng truy vết về context.

Ứng dụng được tổ chức theo nhiều lớp. Web chịu trách nhiệm trình bày và trạng thái tương tác; API thực hiện validation, authentication và điều phối use case; service/module chứa quy tắc nghiệp vụ; Prisma quản lý truy cập quan hệ; còn PostgreSQL, Redis, search và các dịch vụ ngoài đảm nhiệm lưu trữ hoặc năng lực chuyên biệt. Cách phân tách này giúp test từng tầng và thay thế phụ thuộc mà không làm thay đổi toàn bộ giao diện.

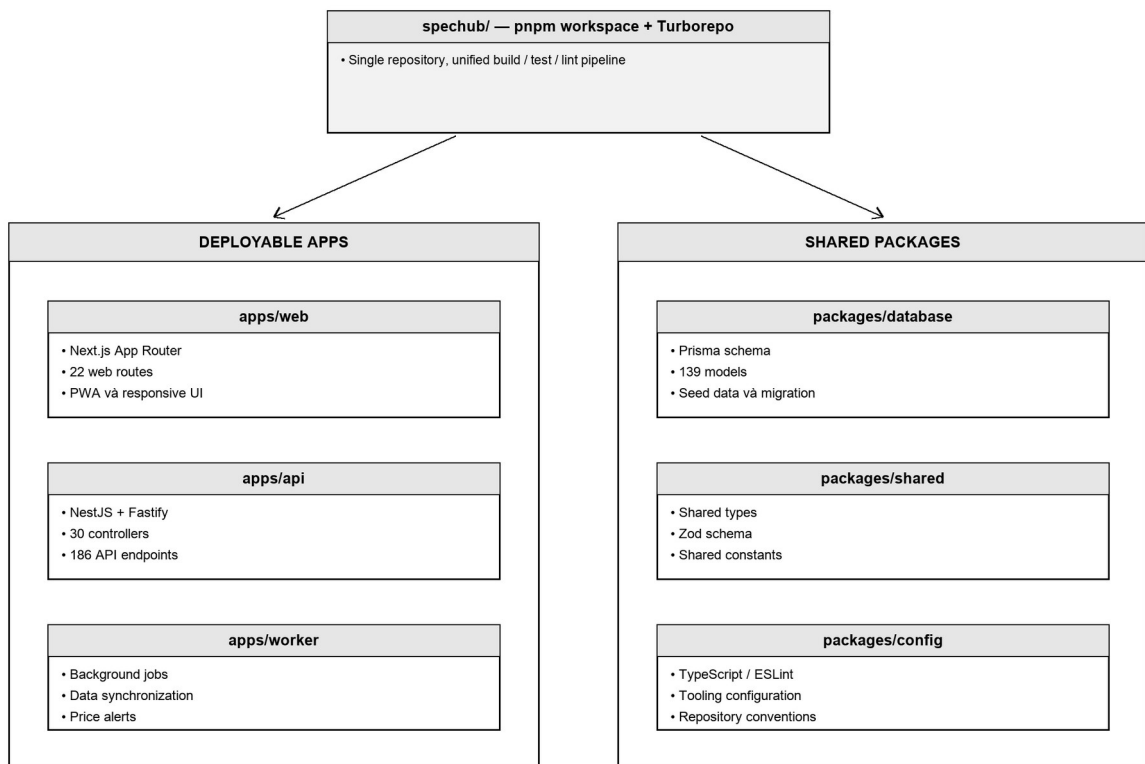
Bảng 2.1. Thành phần công nghệ và phiên bản

Lớp	Công nghệ	Phiên bản	Vai trò
Môi trường thực thi	Node.js	$\geq 22.11.0$	Thực thi web/API và công cụ
Quản lý repository	pnpm / Turborepo	pnpm@9.15.0 / 2.3.3	Quản lý repository đơn nhất và quy trình xử lý
Giao diện web	Next.js / React	15.1.3 / ^19.0.0	SSR/CSR, App Router và giao diện
API	NestJS / Fastify	11.0.6 / 5.2.0	REST, guard, pipe và hiệu năng HTTP
Kiểm tra dữ liệu	Zod	3.24.1	Schema dùng chung và kiểm tra dữ liệu
Truy cập dữ liệu	Prisma	6.1.0	ORM, migration và kiểu dữ liệu
Database	PostgreSQL	16	Quan hệ, tìm kiếm toàn văn, vector và phần mở rộng
Cache/phiên	Redis	Theo môi trường triển khai	Phiên, cache và điều phối
Tìm kiếm	Meilisearch client	0.46.0	Tìm kiếm chuyên dụng tùy chọn

Lớp	Công nghệ	Phiên bản	Vai trò
Trạng thái phía web	TanStack Query	5.62.10	Cache và đồng bộ dữ liệu phía web

2.2. Monorepo và tổ chức mã nguồn

Monorepo dùng pnpm workspace và Turborepo cho phép web, API, worker, database và các shared package phát triển trong một kho nhưng vẫn có ranh giới triển khai. Kiểu dữ liệu, schema validation và cấu hình TypeScript/ESLint được chia sẻ để giảm sai khác giữa client và server. Pipeline có thể chạy build, test và lint theo đồ thị phụ thuộc, tận dụng cache cho phần không thay đổi.



Hình 2.1. Cấu trúc monorepo Spechub

Ranh giới package không thay thế ranh giới nghiệp vụ. Trong API, mỗi miền như catalog, AI, Wiki, alerts, affiliate, billing và admin tiếp tục có module/controller/service riêng. Cách tổ chức hai cấp giúp đội phát triển xác định nơi chứa code, đồng thời hạn chế việc một module truy cập trực tiếp chi tiết nội bộ của module khác.

2.3. Presentation layer với Next.js và React

Next.js App Router tổ chức trang theo route filesystem, hỗ trợ kết xuất server-side cho nội dung cần khả năng indexing và kết xuất phía khách cho tương tác giàu trạng thái. React 19 cung cấp mô hình component; TanStack Query quản lý cache, trạng thái tải/lỗi và làm mới dữ liệu server; Tailwind CSS giúp thống nhất token khoảng cách, màu và responsive.

Danh sách 22 route thể hiện bốn nhóm trải nghiệm: khám phá công khai; tài khoản/cá nhân hóa; nội dung Wiki; và quản trị/thương mại. Thiết kế PWA bổ sung service worker và trang offline để người dùng có phản hồi rõ khi mất kết nối. Việc phân tách route không đồng nghĩa mọi trang đều công khai: middleware và trạng thái authentication phải điều hướng hoặc chặn thao tác cần quyền.

- Server rendering phù hợp trang catalog/chi tiết cần metadata và tải lần đầu ổn định.
- Client component dùng cho bộ lọc, chọn so sánh, biểu mẫu nhiều bước và hội thoại AI.
- TanStack Query giúp tránh tự viết cơ chế cache riêng cho từng trang.
- URL chứa slug và query filter làm trạng thái có thể chia sẻ, quay lại và test.

2.4. API với NestJS và Fastify

NestJS cung cấp dependency injection, module, controller, pipe, guard và interceptor. Fastify được chọn làm HTTP adapter để có pipeline gọn và khả năng xử lý cao. API dùng prefix /api, URI version v1 và DTO được kiểm tra bởi ValidationPipe với whitelist, forbidNonWhitelisted và transform; vì vậy trường không được định nghĩa bị từ chối thay vì âm thầm đi vào nghiệp vụ.

Guard toàn cục xử lý JWT, vai trò và rate limit; decorator công khai cho phép các endpoint đọc như health hoặc tìm kiếm bỏ qua authentication khi phù hợp. Swagger chỉ bật trong môi trường phát triển để hỗ trợ khám phá hợp đồng mà không mở bề mặt không cần thiết ở môi trường sản xuất. Request ID được gắn vào luồng xử lý để liên kết phản hồi lỗi với log.

Thống kê tĩnh ghi nhận 30 controller và 186 endpoint, gồm 91 GET, 57 POST, 24 PATCH, 13 DELETE và 1 PUT. Phân bố này phù hợp hệ thống vừa có bề mặt đọc catalog lớn, vừa có quy trình tạo/duyệt nội dung và tác vụ quản trị.

```
Hợp đồng phản hồi lỗi tham chiếu
HTTP/1.1 400 Bad Request
{
  "statusCode": 400,
  "message": ["field must not be empty"],
  "requestId": "7f6f4e4a-49b4-4c58-a487-3b8ec93260b0"
}
```

2.5. Prisma, PostgreSQL và tìm kiếm dữ liệu

Prisma cung cấp schema khai báo, client có kiểu và quy trình migration. Với 139 model, lợi ích quan trọng là data contract được kiểm tra tập trung và quan hệ được biểu diễn rõ trong code. PostgreSQL 16 phù hợp relational data nhiều ràng buộc, đồng thời cho phép mở rộng tìm kiếm bằng pg_trgm, unaccent và lưu vector bằng pgvector.

Tìm kiếm không chỉ là so khớp chuỗi. Truy vấn cần chuẩn hóa dấu, alias, tên model, hãng, chipset và từ khóa thuộc tính; kết quả phải pagination, xếp hạng và quay lại thực thể đầy đủ. Kiến trúc cho phép dùng khả năng PostgreSQL làm nền và Meilisearch như bộ máy chuyên dụng tùy chọn. Dù đường tìm kiếm nào được chọn, kết quả cuối vẫn được hydrate từ catalog để duy trì một nguồn sự thật.

Vector embedding hỗ trợ retrieval ngữ nghĩa cho Wiki, thông số và câu hỏi AI. Vector không thay thế quan hệ hay citation: nó chỉ giúp chọn context gần nghĩa. Thực thể gốc, phiên bản, model embedding và liên kết nguồn cần được lưu để có thể tái tạo hoặc vô hiệu hóa index khi dữ liệu thay đổi.

2.6. Redis, background job và resilience

Redis được sử dụng cho session, cache và các trạng thái ngắn hạn cần truy cập nhanh. Refresh token gắn với session UUID lưu trong Redis giúp logout thực sự thu hồi phiên, thay vì chỉ xóa token ở trình duyệt. Cache phải có TTL và namespace theo phiên bản để tránh trả dữ liệu cũ sau khi catalog được xuất bản.

Worker xử lý cảnh báo giá, đồng bộ nguồn, indexing hoặc gửi thông báo—những tác vụ không nên giữ request HTTP mở. Thiết kế job cần idempotent, có khóa chống chạy

trùng, retry giới hạn và dead-letter/ghi lỗi rõ. Readiness kiểm tra DB và Redis giúp bộ điều phối chỉ gửi lưu lượng khi phụ thuộc cốt lõi sẵn sàng.

2.7. Retrieval-Augmented Generation (RAG) và AI có citation

RAG kết hợp bước retrieval context với mô hình generation. Câu hỏi được chuẩn hóa, tìm top-k đoạn hoặc thực thể phù hợp, sau đó API xây dựng prompt gồm chính sách trả lời, context và định danh citation. Mô hình tạo câu trả lời; hệ thống kiểm tra định dạng và trả cả citation để giao diện hiển thị liên kết nguồn.

Ba ranh giới kiểm soát chất lượng là retrieval, generation và presentation. Retrieval phải lọc theo trạng thái xuất bản và quyền; generation không được coi kiến thức ngoài context là dữ kiện chắc chắn; presentation cần phân biệt câu trả lời, cảnh báo thiếu bằng chứng và nguồn. Cache AI nên gắn với hash truy vấn, phiên bản dữ liệu và model để tránh dùng lại kết quả sau thay đổi quan trọng.

Đánh giá AI không thể chỉ dùng test unit. Cần tập câu hỏi chuẩn, đáp án/nguồn mong đợi, thước đo citation precision, context recall, mức phù hợp và kiểm tra an toàn prompt injection. Phạm vi hiện tại đã có kiến trúc và endpoint, nhưng bộ benchmark định lượng toàn diện là hướng phát triển tiếp theo.

2.8. Authentication, authorization và security

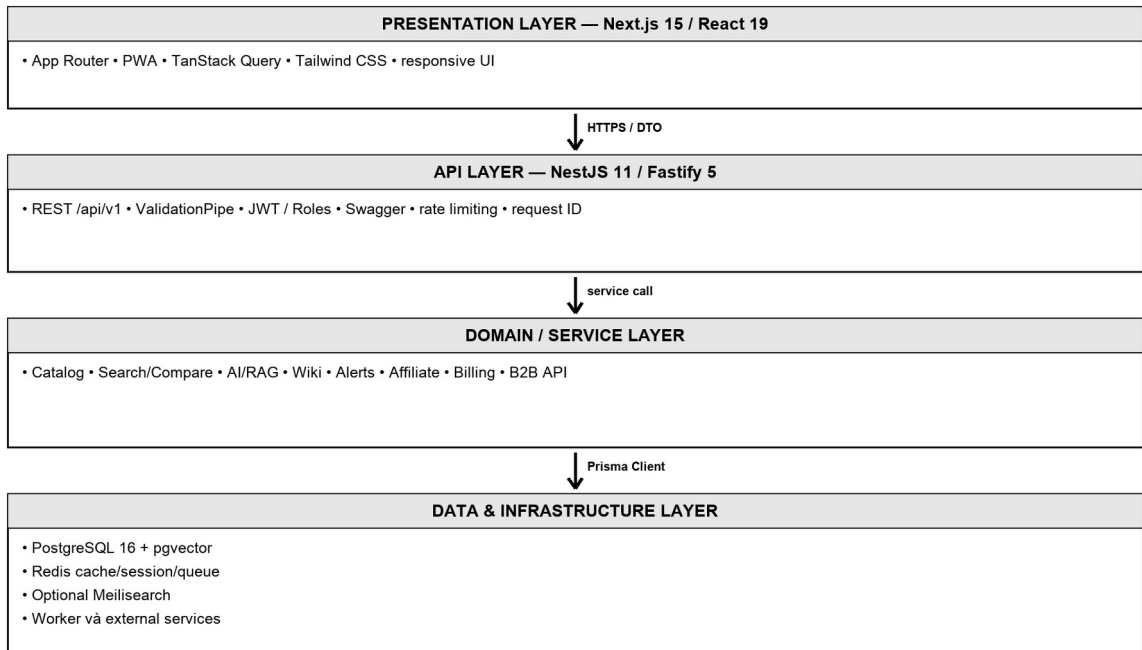
Luồng authentication phát hành access token ngắn hạn và refresh token gắn phiên. Mật khẩu được lưu dưới dạng băm; refresh cookie cần HttpOnly, Secure và SameSite phù hợp. RBAC phân biệt USER, EDITOR, MODERATOR và ADMIN; quyền chỉnh sửa catalog hoặc xuất bản Wiki được kiểm tra ở server, không dựa vào việc ấn nút trên giao diện.

Bảng 2.2. Biện pháp security theo lớp

Lớp	Rủi ro	Biện pháp
Trình duyệt	XSS, đánh cắp token	Escape nội dung, CSP/Helmet, refresh cookie HttpOnly, không lưu bí mật trong localStorage
HTTP/API	Payload lạ, abuse	HTTPS, CORS giới hạn, ValidationPipe, giới hạn kích thước và rate limit

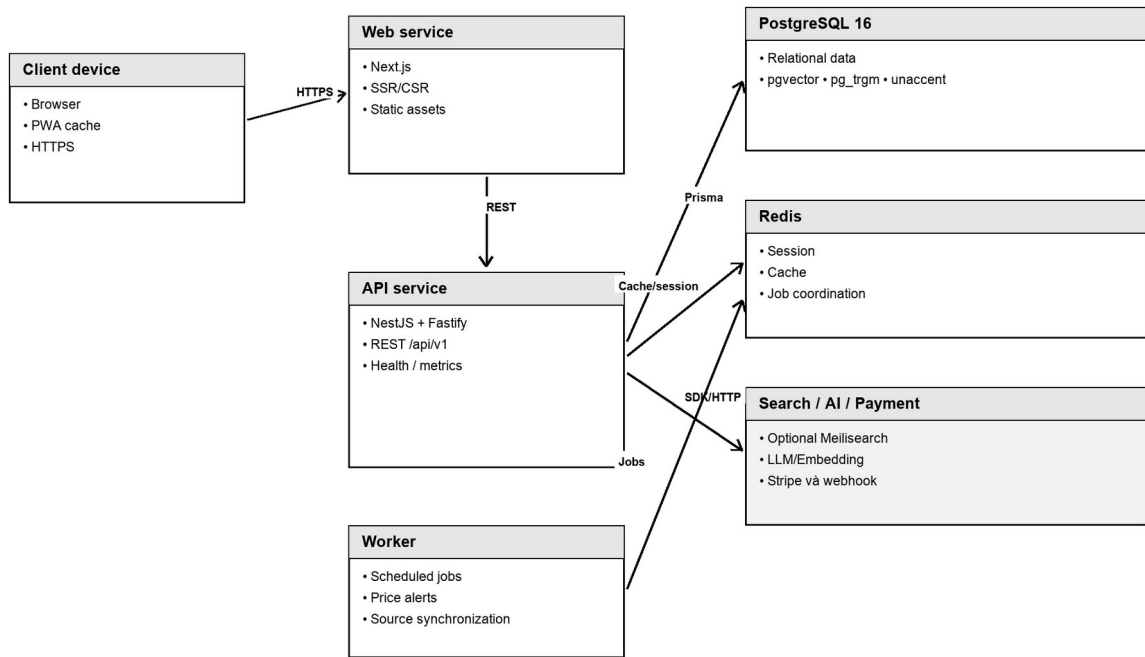
Lớp	Rủi ro	Biện pháp
Authentication	Credential stuffing, phiên tồn tại	Hash mật khẩu, throttle, session Redis, rotate/thu hồi refresh
Authorization	Leo thang đặc quyền	JWT guard, Roles guard, deny-by-default và kiểm tra ownership
Dữ liệu	Injection, mất toàn vẹn	Prisma parameterization, ràng buộc DB, transaction, migration và backup
Webhook/API key	Replay, lộ khóa	Xác minh chữ ký, idempotency, lưu hash, scope và revoked_at
AI/RAG	Prompt injection, lộ dữ liệu	Lọc nguồn, tách system policy, không retrieval tài liệu vượt quyền, citation bắt buộc
Vận hành	Thiếu truy vết	Request ID, audit log, metrics, cảnh báo và scrub dữ liệu nhạy cảm

2.9. PWA, observability và triển khai



Hình 2.2. Kiến trúc logic nhiều lớp

PWA cải thiện khả năng cài đặt và phản hồi khi kết nối không ổn định, nhưng cache phải được thiết kế cẩn trọng để không làm stale dữ liệu giá hoặc trạng thái tài khoản. Observability gồm health, liveness, readiness, metrics, structured log và request ID; audit log tập trung vào thay đổi có ảnh hưởng nghiệp vụ. Hai lớp này phục vụ các mục đích khác nhau và không nên trộn lẫn.



Hình 2.3. Mô hình triển khai tham chiếu

Bảng 2.3. Lựa chọn kiến trúc và đánh đổi

Lựa chọn	Lợi ích	Đánh đổi / kiểm soát
Monorepo	Chia sẻ kiểu và pipeline	Kho lớn; cần ranh giới package và cache build
Modular monolith API	Giao dịch và triển khai đơn giản	Cần kỷ luật phụ thuộc để tránh module chồng chéo
PostgreSQL trung tâm	Toàn vẹn quan hệ và truy vấn linh hoạt	Schema lớn; cần index, migration và ownership rõ
Redis	Phiên/cache nhanh	Thêm phụ thuộc; phải thiết kế TTL, eviction và phục hồi
Meilisearch tùy chọn	Tìm kiếm xếp hạng tốt	Đồng bộ index và chế độ fallback
RAG	AI bám dữ liệu và có citation	Chi phí, latency và yêu cầu benchmark chất lượng
Worker	Tách tác vụ dài khỏi request	Cần idempotency, retry, giám sát và xử lý lỗi

2.10. Kết luận chương

Các công nghệ được lựa chọn tạo thành một kiến trúc TypeScript xuyên suốt, có relational data làm nền, cache và search engine là các năng lực hỗ trợ, còn AI được đặt sau

lớp retrieval và citation. Điểm quan trọng không nằm ở danh sách framework mà ở cách các ranh giới và cơ chế kiểm soát được ghép thành những luồng có thể test.

Next.js và React đảm nhiệm presentation layer; NestJS/Fastify tổ chức hợp đồng HTTP và chính sách truy cập; Prisma/PostgreSQL bảo đảm tính toàn vẹn của dữ liệu nghiệp vụ; Redis hỗ trợ phiên, cache và điều phối ngăn chặn. Worker tiếp nhận các công việc dài hoặc cần retry, qua đó giữ thời gian phản hồi của API trong giới hạn có thể kiểm soát. Cách phân vai này phù hợp với cấu trúc monorepo vì cho phép chia sẻ kiểu dữ liệu nhưng vẫn duy trì quyền sở hữu theo miền.

Đối với tìm kiếm và AI, thiết kế ưu tiên dữ liệu đã xuất bản, nguồn có thể kiểm chứng và cơ chế suy giảm có kiểm soát. Kết quả sinh không được xem là dữ liệu gốc; hệ thống cần lưu context, phiên bản index, citation và dấu vết đánh giá. Đối với security, authentication chỉ là lớp đầu; quyết định truy cập còn phải dựa trên vai trò, phạm vi tài nguyên, trạng thái nội dung, rate limit và audit log.

Các lựa chọn trên tạo ra cơ sở kỹ thuật để chuyển yêu cầu nghiệp vụ thành mô hình thiết kế có thể triển khai. Chương 3 tiếp tục cụ thể hóa bằng tác nhân, use case, sequence diagram, quy trình nền, hợp đồng API và các ERD theo miền dữ liệu.

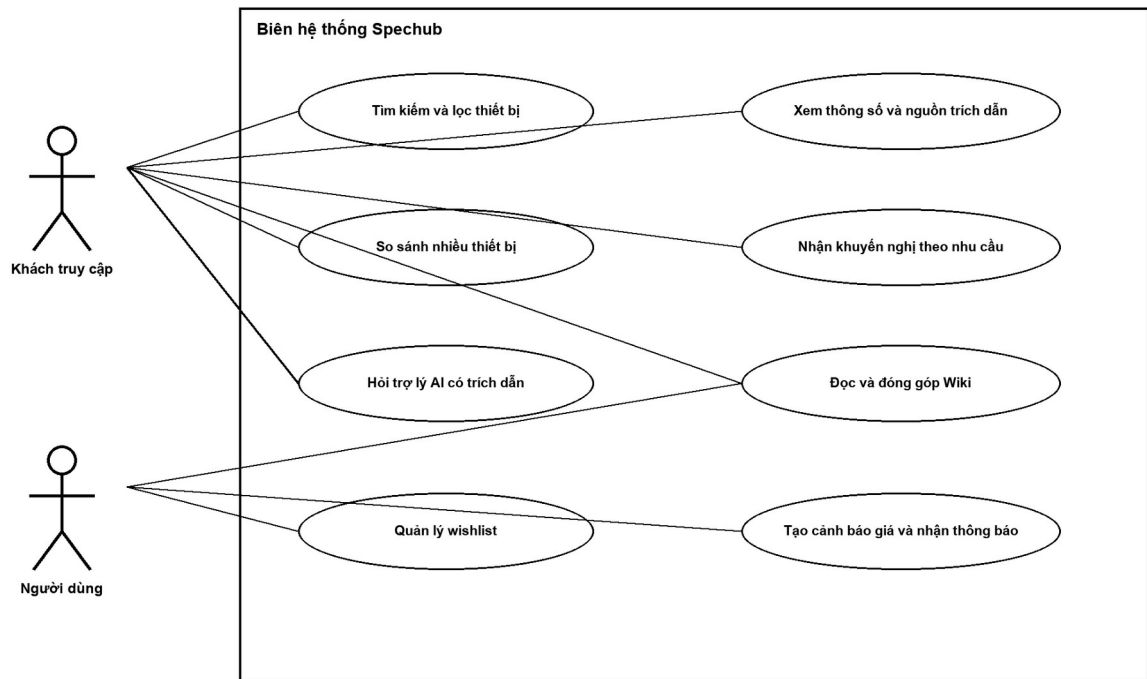
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Tác nhân và mô hình use case

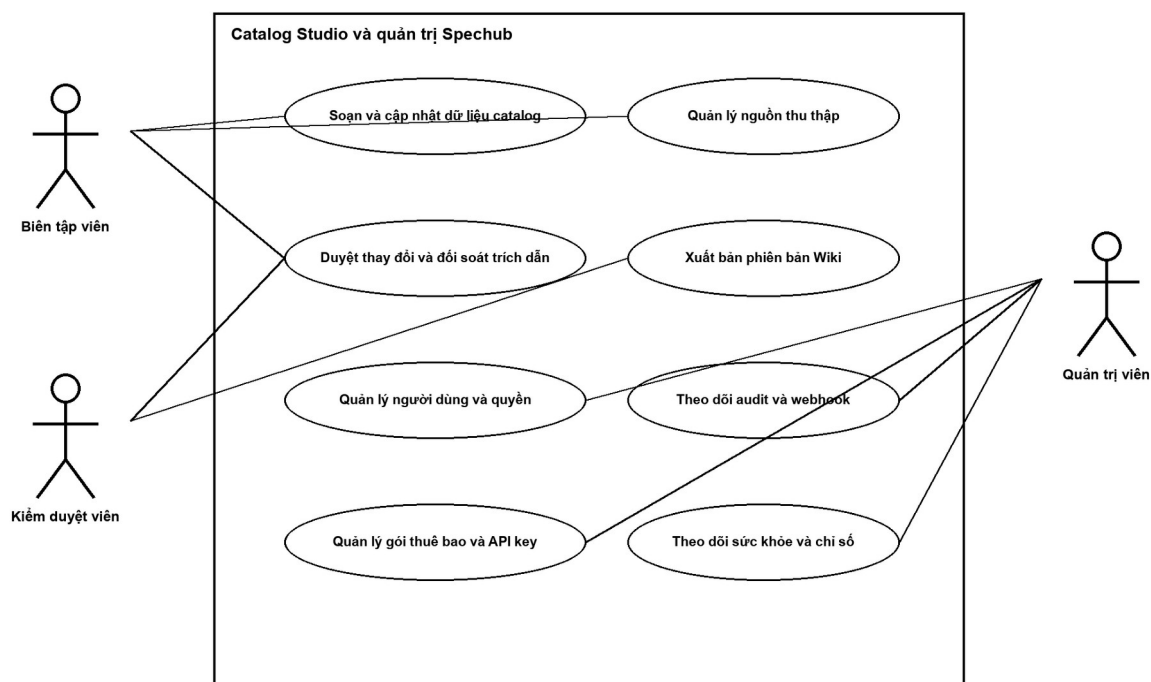
Phân tích tác nhân dựa trên quyền và mục tiêu nghiệp vụ, không đồng nhất tác nhân với một màn hình. Khách truy cập có thể thực hiện luồng đọc; người dùng kế thừa quyền đọc và có dữ liệu cá nhân; ba vai trò nội bộ có đặc quyền tăng dần; đối tác B2B tương tác bằng API key thay vì phiên trình duyệt.

Bảng 3.1. Tác nhân hệ thống

Tác nhân	Authentication	Khả năng chính	Giới hạn
Khách truy cập	Không bắt buộc	Tìm kiếm, lọc, xem chi tiết, so sánh, khuyến nghị công khai	Không lưu dữ liệu cá nhân lâu dài
Người dùng	JWT + refresh session	Wishlist, cảnh báo, thông báo, đóng góp Wiki	Chỉ thao tác dữ liệu của mình; nội dung chờ duyệt
Biên tập viên	JWT, role EDITOR	Soạn catalog, quản lý nguồn, đối soát dữ liệu	Không quản trị người dùng/hệ thống ngoài phạm vi
Kiểm duyệt viên	JWT, role MODERATOR	Duyệt thay đổi và phiên bản Wiki	Quyết định được ghi audit
Quản trị viên	JWT, role ADMIN	Quản trị quyền, gói, API key, webhook và vận hành	Thao tác nhạy cảm phải có log và xác nhận
Đối tác B2B	API key + scope	Đọc dữ liệu theo hợp đồng và hạn mức	Không dùng endpoint nội bộ; có quota và thu hồi
Worker	Danh tính dịch vụ	Đồng bộ, cảnh báo, thông báo, indexing	Không nhận tương tác trực tiếp; job phải idempotent



Hình 3.1. Use case phía người dùng



Hình 3.2. Use case biên tập và quản trị

Bảng 3.2. Danh mục use case

Mã	Tên	Tác nhân chính	Kết quả
UC-01	Tìm kiếm và lọc thiết bị	Khách/Người dùng	Danh sách xếp hạng, pagination và có bộ lọc
UC-02	So sánh thiết bị	Khách/Người dùng	Ma trận thuộc tính chuẩn hóa
UC-03	Khuyến nghị và hỏi AI	Khách/Người dùng	Gợi ý/câu trả lời có lý do và citation
UC-04	Đăng nhập và quản lý phiên	Người dùng nội bộ	Access token và refresh session có thể thu hồi
UC-05	Wishlist và cảnh báo giá	Người dùng	Theo dõi variant và notification khi đạt ngưỡng
UC-06	Đóng góp và duyệt Wiki	Người dùng/Moderator	Revision được xuất bản hoặc từ chối có lý do
UC-07	Thu thập và duyệt catalog	Editor/Moderator/Worker	Dữ liệu chuẩn hóa có nguồn được xuất bản
UC-08	Khai thác API B2B	Đối tác/Admin	Truy cập theo scope, quota và usage

3.2. Đặc tả use case

Tám use case dưới đây đại diện cho hành trình đọc, cá nhân hóa, cộng tác, quản trị dữ liệu và tích hợp. Luồng được viết ở mức nghiệp vụ; đường dẫn API cụ thể có thể thay đổi nhưng tiền/hậu điều kiện và kiểm soát phải được giữ.

Bảng 3.3. Đặc tả UC-01 – Tìm kiếm và lọc thiết bị

Thuộc tính	Nội dung
Mã use case	UC-01
Mục tiêu	Tìm nhanh thiết bị hoặc phần cứng phù hợp theo từ khóa, loại, hãng và thuộc tính.
Tác nhân	Khách truy cập; Người dùng.
Tiền điều kiện	Catalog có dữ liệu đã xuất bản; dịch vụ tìm kiếm ở chế độ PostgreSQL hoặc Meilisearch sẵn sàng.
Kích hoạt	Tác nhân nhập từ khóa, chọn bộ lọc hoặc mở URL có query parameter.
Luồng chính	1) Web chuẩn hóa input và gửi GET search. 2) API kiểm tra tham số, giới hạn trang và sort. 3) Search tìm ID xếp hạng theo text/alias. 4) Catalog hydrate model, variant, media và facet. 5) API trả DTO pagination. 6) Web hiển thị kết quả, bộ lọc và trạng thái URL.
Luồng thay thế / ngoại lệ	A1) Từ khóa rỗng: trả danh mục theo filter. A2) Không có kết quả: gợi ý bỏ bớt filter hoặc alias gần đúng. E1) Search chuyên dụng lỗi: fallback sang PostgreSQL nếu cấu hình cho phép. E2) Tham số sai: trả 400 cùng request ID.
Hậu điều kiện	Không thay đổi catalog; có thể ghi search log và latency mà không lưu dữ liệu nhạy cảm.
Dữ liệu và kiểm soát	Whitelist filter; giới hạn page size; chỉ trả bản ghi published; query được parameter hóa; cache theo query chuẩn hóa.

Bảng 3.4. Đặc tả UC-02 – So sánh thiết bị

Thuộc tính	Nội dung
Mã use case	UC-02
Mục tiêu	Đặt từ hai đến một số hữu hạn thiết bị cạnh nhau trên cùng hệ thuộc tính.
Tác nhân	Khách truy cập; Người dùng.
Tiền điều kiện	Các slug hợp lệ và model có ít nhất một variant được xuất bản.
Kích hoạt	Tác nhân chọn nút So sánh từ danh sách/chi tiết hoặc mở URL compare.
Luồng chính	1) Web duy trì danh sách slug không trùng. 2) API nhận danh sách và kiểm tra số lượng. 3) Service tải model, variant và phần cứng liên quan. 4) Thuộc tính được ánh xạ vào schema so sánh theo category. 5) Giá trị được chuẩn hóa đơn vị và đánh dấu thiếu dữ liệu. 6) Web hiển thị ma trận, điểm khác biệt và nguồn khi có.
Luồng thay thế / ngoại lệ	- A1) Chỉ có một thiết bị: yêu cầu chọn thêm. - A2) Khác category không tương thích: dùng tập thuộc tính chung hoặc cảnh báo. - E1) Slug không tồn tại: bỏ mục lỗi và thông báo. - E2) Dữ liệu thiếu: hiển thị 'Chưa có dữ liệu', không suy diễn.
Hậu điều kiện	URL chứa lựa chọn có thể chia sẻ; không thay đổi dữ liệu người dùng nếu chưa lưu.
Dữ liệu và kiểm soát	Giới hạn số model; thứ tự ổn định; chuẩn hóa đơn vị; citation gắn thuộc tính; tránh N+1 bằng include/batch.

Bảng 3.5. Đặc tả UC-03 – Nhận khuyến nghị và hỏi AI

Thuộc tính	Nội dung
Mã use case	UC-03
Mục tiêu	Chuyển nhu cầu tự nhiên hoặc bộ tiêu chí thành gợi ý có thể giải thích và câu trả lời có nguồn.
Tác nhân	Khách truy cập; Người dùng; AI service.
Tiền điều kiện	Catalog/Wiki đã indexing; chính sách prompt và cấu hình model hợp lệ.
Kích hoạt	Tác nhân hoàn thành biểu mẫu khuyến nghị hoặc gửi câu hỏi.
Luồng chính	1) API validate nhu cầu/câu hỏi. 2) Bộ retrieval tạo query và lấy top-k context theo quyền. 3) Catalog tải dữ kiện/citation tương ứng. 4) AI service tạo prompt ràng buộc. 5) LLM trả nội dung có tham chiếu ID. 6) API kiểm tra, lưu log/cache phù hợp và trả câu trả lời cùng citation. 7) Web hiển thị kết quả, lý do và liên kết nguồn.
Luồng thay thế / ngoại lệ	A1) Không đủ context: trả lời giới hạn và đề nghị làm rõ. A2) Model ngoài danh mục: nêu rõ không tìm thấy. E1) Dịch vụ LLM lỗi: trả thông báo có thể retry, không giả câu trả lời. E2) Citation không hợp lệ: loại phần không kiểm chứng hoặc đánh dấu.
Hậu điều kiện	Có bản ghi đo chất lượng/latency nếu chính sách cho phép; catalog không bị sửa bởi câu trả lời.
Dữ liệu và kiểm soát	Giới hạn độ dài; lọc prompt injection; chỉ context published; timeout; cache theo phiên bản; citation bắt buộc cho dữ kiện.

Bảng 3.6. Đặc tả UC-04 – Đăng nhập và quản lý phiên

Thuộc tính	Nội dung
Mã use case	UC-04
Mục tiêu	Authentication an toàn và cho phép thu hồi phiên độc lập với access token.
Tác nhân	Người dùng; Biên tập viên; Moderator; Admin.
Tiền điều kiện	Tài khoản hoạt động; PostgreSQL và Redis sẵn sàng.
Kích hoạt	Tác nhân gửi email/mật khẩu hoặc access token hết hạn cần làm mới.
Luồng chính	1) API validate credential. 2) Tìm user và so password hash. 3) Tạo session UUID trong Redis với TTL. 4) Phát access token ngắn hạn và refresh token. 5) Web nhận hồ sơ và lưu trạng thái. 6) Khi refresh, API xác minh chữ ký và session. 7) Logout xóa/thu hồi session và cookie.
Luồng thay thế / ngoại lệ	E1) Sai credential: phản hồi chung, không tiết lộ email tồn tại. E2) Session hết hạn/thu hồi: yêu cầu đăng nhập lại. E3) Tài khoản khóa: từ chối và audit. E4) Redis không sẵn sàng: readiness fail; không cấp phiên không thể quản lý.
Hậu điều kiện	Phiên hợp lệ được lưu có TTL hoặc đã bị thu hồi; sự kiện đăng nhập đặc quyền có thể được audit.
Dữ liệu và kiểm soát	Rate limit; hash mật khẩu; cookie HttpOnly/Secure; rotate token; RBAC server-side; scrub token khỏi log.

Bảng 3.7. Đặc tả UC-05 – Wishlist và cảnh báo giá

Thuộc tính	Nội dung
Mã use case	UC-05
Mục tiêu	Lưu variant quan tâm và nhận thông báo khi giá đạt điều kiện.
Tác nhân	Người dùng; Worker; Notification service.
Tiền điều kiện	Người dùng đăng nhập; variant tồn tại; kênh thông báo được cấu hình.
Kích hoạt	Người dùng thêm wishlist hoặc đặt target price.
Luồng chính	1) API kiểm tra ownership, variant, tiền tệ và ngưỡng. 2) Lưu wishlist/alert duy nhất. 3) Worker định kỳ đọc giá mới. 4) So khớp điều kiện và chống gửi lặp. 5) Tạo notification và delivery. 6) Gửi qua kênh được bật. 7) Ghi trạng thái delivery và lần kích hoạt.
Luồng thay thế / ngoại lệ	A1) Giá đã dưới ngưỡng tại lúc tạo: thông báo ngay theo chính sách. A2) Người dùng tắt alert: worker bỏ qua. E1) Kênh gửi lỗi: retry giới hạn, ghi failed. E2) Giá thiếu/stale: không kích hoạt, ghi lý do.
Hậu điều kiện	Alert có trạng thái và last_triggered_at; delivery có lịch sử để chống lặp và hỗ trợ.
Dữ liệu và kiểm soát	Unique user+variant; decimal/currency chuẩn; idempotency key; quiet hours; preference và unsubscribe.

Bảng 3.8. Đặc tả UC-06 – Đóng góp và duyệt Wiki

Thuộc tính	Nội dung
Mã use case	UC-06
Mục tiêu	Cho phép cộng tác nội dung mà vẫn duy trì lịch sử, nguồn và quyền xuất bản.
Tác nhân	Người dùng; Biên tập viên; Kiểm duyệt viên.
Tiền điều kiện	Tác nhân đăng nhập; bài viết hoặc quyền tạo mới hợp lệ.
Kích hoạt	Tác nhân tạo/sửa bài Wiki và gửi duyệt.
Luồng chính	1) Web tải current revision và citation. 2) Tác nhân soạn nội dung, thêm nguồn. 3) API validate và tạo revision draft. 4) Gửi revision vào queue review. 5) Moderator xem diff, nguồn và bình luận. 6) Phê duyệt tạo current revision mới hoặc từ chối có lý do. 7) Sự kiện xuất bản làm mới search/embedding.
Luồng thay thế / ngoại lệ	A1) Xung đột revision: yêu cầu merge trên phiên bản mới. A2) Yêu cầu bổ sung nguồn: trả về draft. E1) Citation không hợp lệ: không cho xuất bản. E2) Tác nhân thiếu role: trả 403 và audit.
Hậu điều kiện	Mọi revision được giữ; bài published trở current_revision; quyết định review có actor và thời gian.
Dữ liệu và kiểm soát	Optimistic concurrency; sanitize rich text; RBAC; audit; không sửa trực tiếp revision đã xuất bản.

Bảng 3.9. Đặc tả UC-07 – Thu thập và duyệt catalog

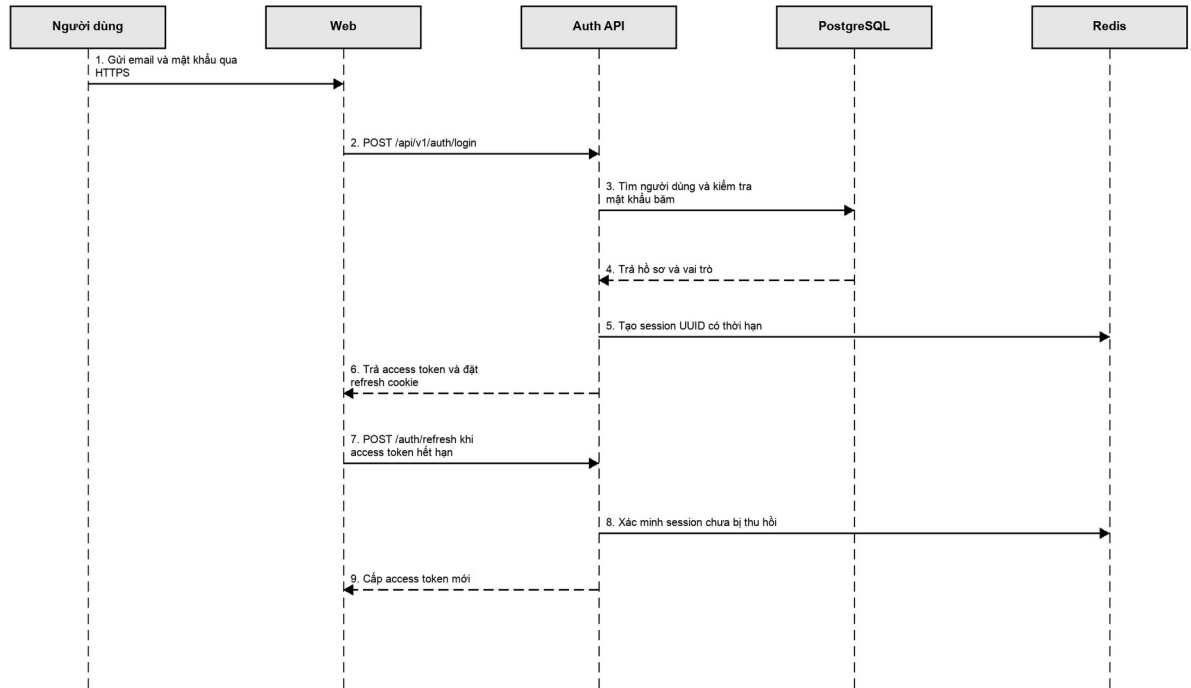
Thuộc tính	Nội dung
Mã use case	UC-07
Mục tiêu	Đưa dữ liệu nguồn vào catalog qua pipeline có kiểm soát và truy nguyên.
Tác nhân	Worker; Biên tập viên; Kiểm duyệt viên.
Tiền điều kiện	Data source được đăng ký, có chính sách thu thập và parser phù hợp.
Kích hoạt	Lịch job, webhook hoặc thao tác đồng bộ thủ công.
Luồng chính	1) Worker fetch và lưu raw page/content hash. 2) Parser trích xuất candidate. 3) Chuẩn hóa alias, đơn vị và quan hệ. 4) So với phiên bản catalog, tạo diff kèm citation. 5) Editor đối soát và chỉnh candidate. 6) Moderator phê duyệt/từ chối. 7) Transaction xuất bản, audit và phát sự kiện reindex.
Luồng thay thế / ngoại lệ	A1) Content hash không đổi: kết thúc không tạo diff. A2) Nguồn xung đột: giữ candidate và yêu cầu nguồn thứ hai. E1) Parser lỗi: lưu raw page/log, không tác động catalog. E2) Transaction lỗi: rollback toàn bộ.
Hậu điều kiện	Catalog chỉ thay đổi khi được duyệt; nguồn, raw page, diff và quyết định được lưu.
Dữ liệu và kiểm soát	Rate/robots policy; hash; schema validation; transaction; idempotency; audit và quyền theo vai trò.

Bảng 3.10. Đặc tả UC-08 – Khai thác API B2B

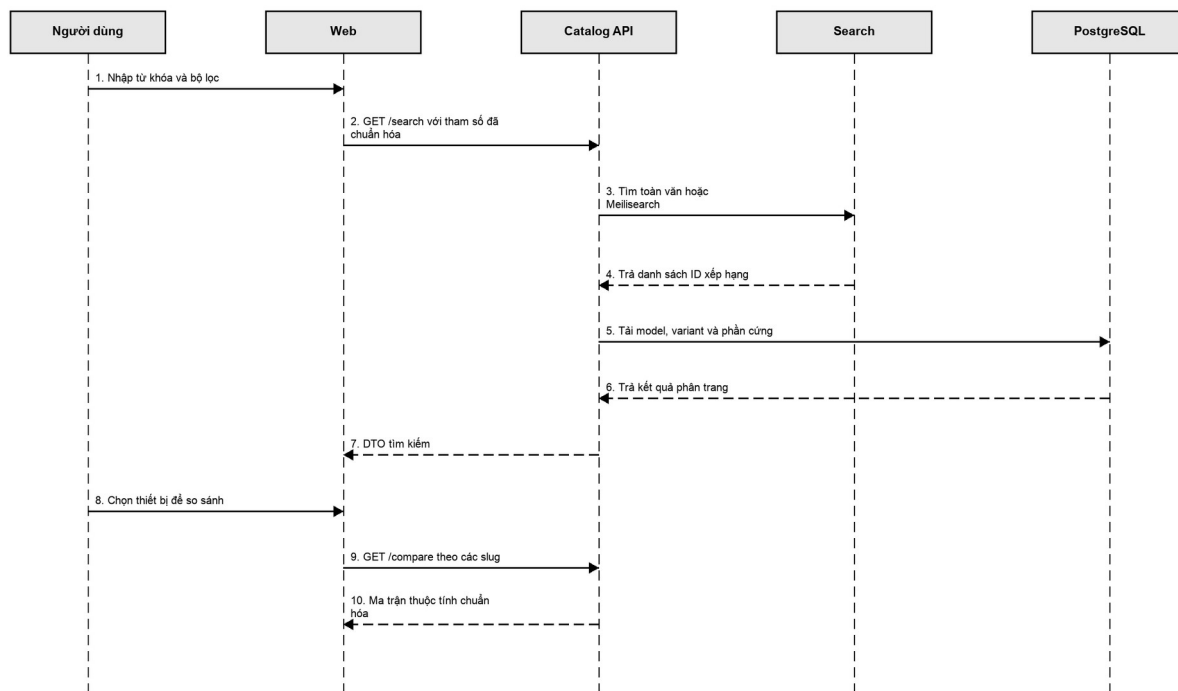
Thuộc tính	Nội dung
Mã use case	UC-08
Mục tiêu	Cho đối tác truy cập dữ liệu theo scope, gói và hạn mức có thể theo dõi.
Tác nhân	Đối tác B2B; Quản trị viên.
Tiền điều kiện	Đối tác có subscription/API key hoạt động và scope phù hợp.
Kích hoạt	Đối tác gửi request có API key tới endpoint cho phép.
Luồng chính	1) Gateway/guard băm key và tìm bản ghi. 2) Kiểm tra revoked_at, subscription, scope và quota. 3) Validate tham số và gọi service catalog. 4) Trả DTO versioned. 5) Ghi usage theo đơn vị và route. 6) Trả header quota/request ID. 7) Admin xem usage hoặc thu hồi/rotate key.
Luồng thay thế / ngoại lệ	E1) Key sai/thu hồi: 401. E2) Thiếu scope: 403. E3) Vượt quota: 429 và retry guidance. E4) Gói hết hạn: chặn và phát sự kiện billing nếu cần.
Hậu điều kiện	Usage được cộng idempotent; không lưu key dạng rõ; đối tác có thể quan sát hạn mức.
Dữ liệu và kiểm soát	Key chỉ hiển thị một lần; lưu hash; scope tối thiểu; rate limit; audit rotate/revoke; API versioning.

3.3. Thiết kế sequence diagram

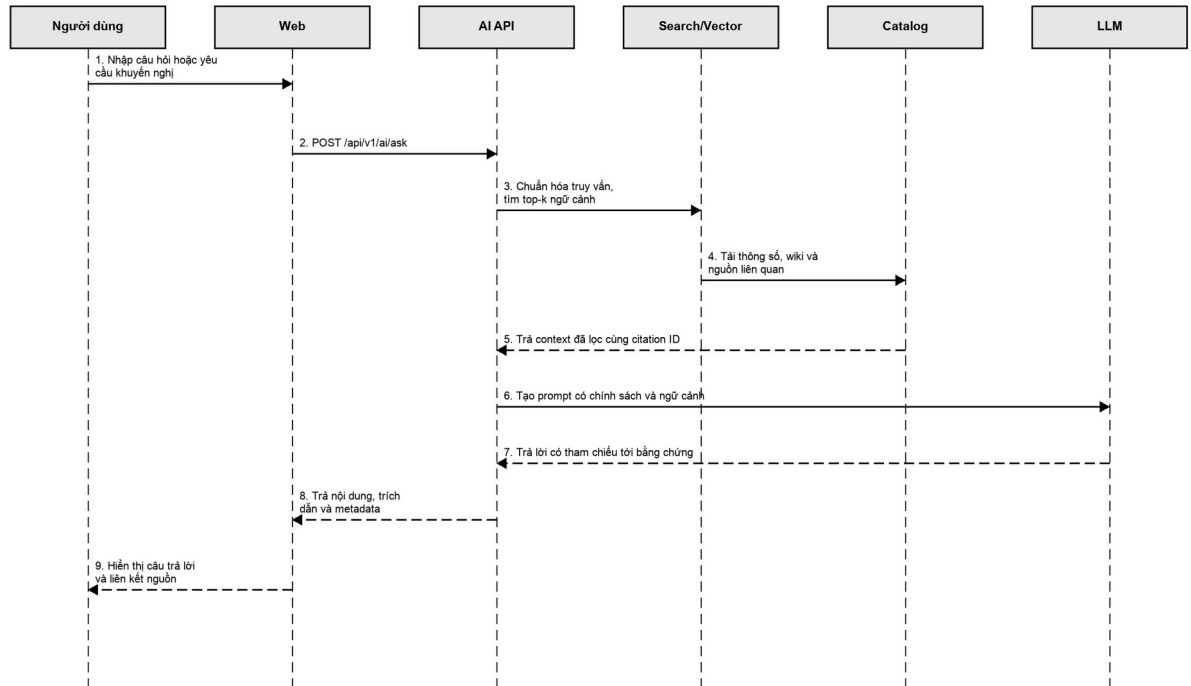
Sequence diagram tập trung vào trust boundary và nơi dữ liệu được validate. Login flow cho thấy refresh token không tự đủ để cấp session mới; Redis session là điều kiện bắt buộc. Search flow tách ranking ID khỏi hydrate entity. AI flow tách retrieval, catalog và generation để giữ citation.



Hình 3.3. Sequence diagram đăng nhập và refresh session



Hình 3.4. Sequence diagram tìm kiếm và so sánh

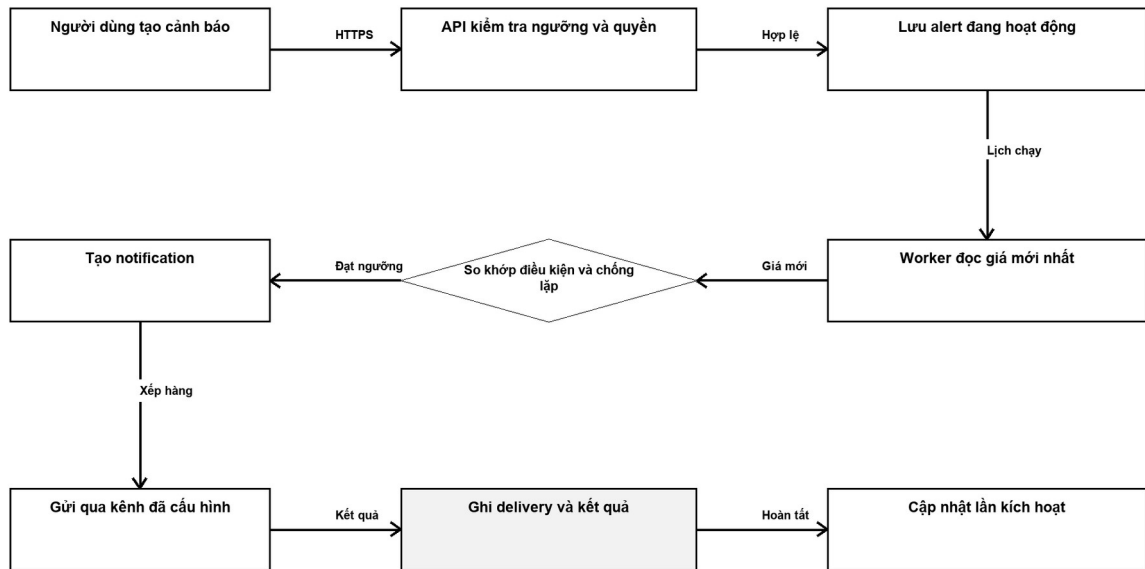


Hình 3.5. Sequence diagram trợ lý AI theo RAG

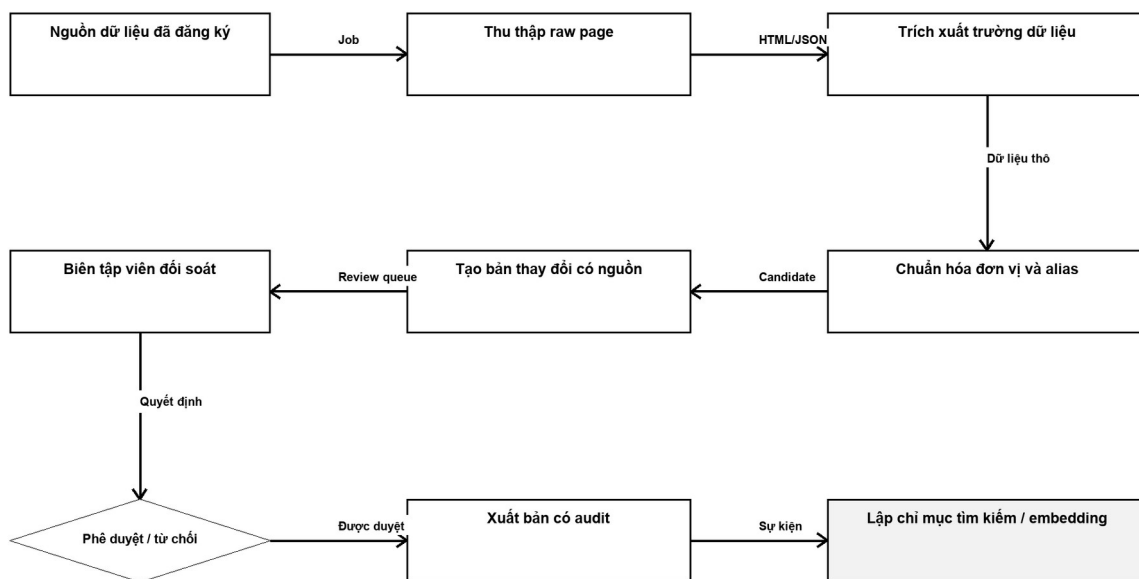
Điểm chung của ba luồng là Web không truy cập dữ liệu nền trực tiếp. API chịu trách nhiệm validate, authorize và tạo DTO; các dịch vụ hạ tầng trả dữ liệu kỹ thuật, còn quyết định nghiệp vụ nằm trong module. Nhờ đó, client không cần biết cấu trúc 139 bảng và không thể bỏ qua quy tắc bằng cách gọi DB/search trực tiếp.

3.4. Thiết kế background workflow

Hai background workflow có state và yêu cầu idempotency. Price alert flow phải phân biệt current price với threshold event, tôn trọng user preference và lưu delivery attempt. Ingestion pipeline phải giữ raw page ngay cả khi parser thất bại để hỗ trợ debugging và reprocessing.



Hình 3.6. Price alert workflow



Hình 3.7. Ingestion workflow

3.5. Thiết kế dữ liệu

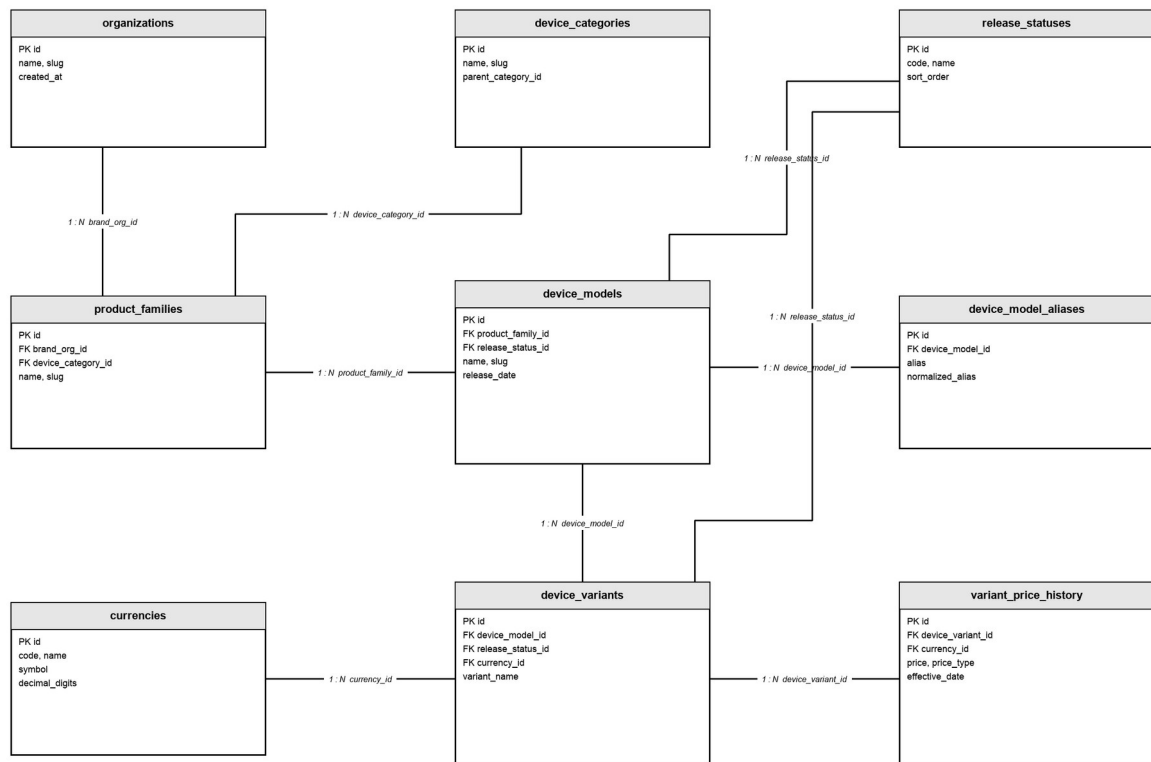
Schema Prisma được phân tích có 139 model. Một ERD duy nhất với toàn bộ bảng sẽ không đọc được trên khổ A4, vì vậy báo cáo chia thành bốn miền và chỉ hiển thị thực thể/thuộc tính tiêu biểu. Danh sách vật lý đầy đủ được đưa vào Phụ lục B. Các bảng dùng snake_case và quan hệ được định nghĩa rõ trong Prisma.

Nguyên tắc thiết kế là giữ thuộc tính cốt lõi ở cột có kiểu, dùng join table cho quan hệ N–N, tách dữ liệu lịch sử/phiên bản khỏi bản hiện hành, và tách citation khỏi thực thể để một nguồn có thể chứng minh nhiều mệnh đề. JSON chỉ phù hợp metadata linh hoạt, không thay thế foreign key cho quan hệ cần truy vấn hoặc ràng buộc.

Bảng 3.11. Phân nhóm 139 data model

Miền	Nhóm thực thể	Mục đích
Core catalog	organizations, device_categories, product_families, device_models, device_variants	Định danh sản phẩm và variant chuẩn hóa
Hardware/scoring	chipsets, display_units, join tables, scoring_profiles, scorecards, benchmarks	Mô tả component, cấu hình và kết quả đo

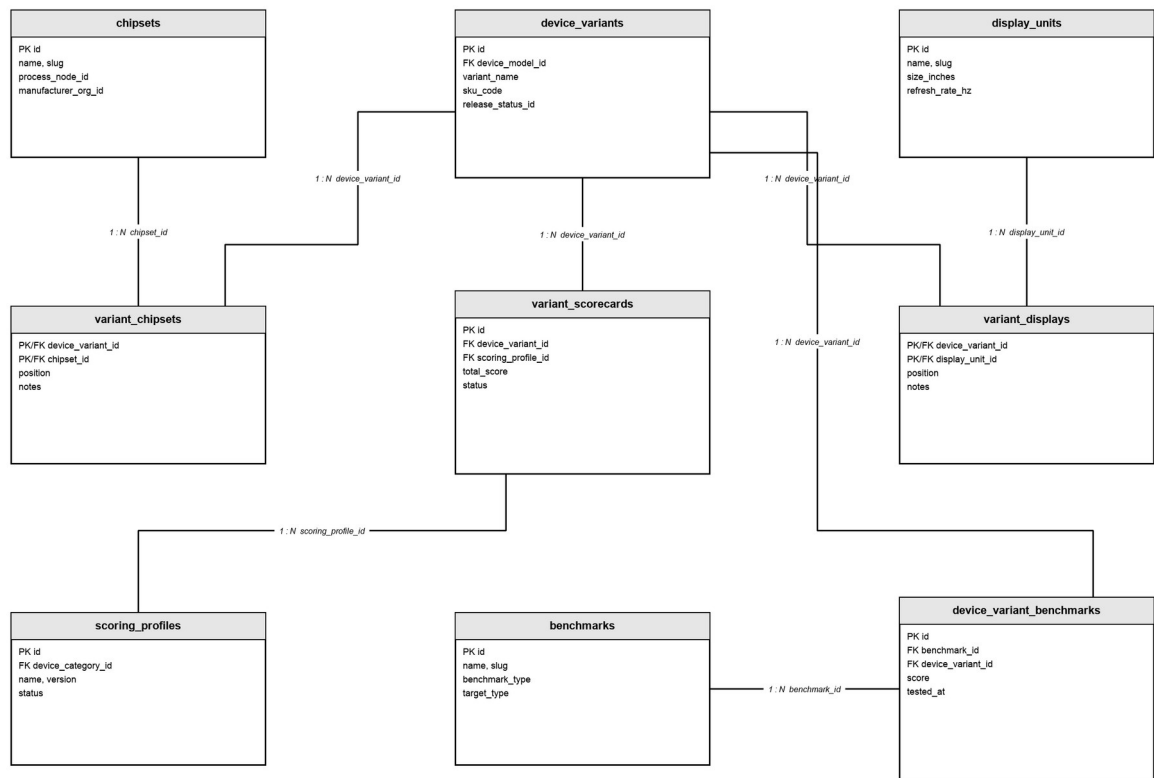
Miền	Nhóm thực thể	Mục đích
Content/crawler/AI	sources, citations, Wiki, revision, raw_pages, embeddings, AI query cache, search logs	Traceability, cộng tác và semantic retrieval
User engagement/commerce	users, wishlists, alerts, notifications, subscriptions, API keys, affiliate, webhooks	Cá nhân hóa, doanh thu và vận hành



Ký hiệu: 1 : N = one-to-many; nét đứt = logical reference, không phải physical foreign key.

Hình 3.8. ERD nhóm catalog

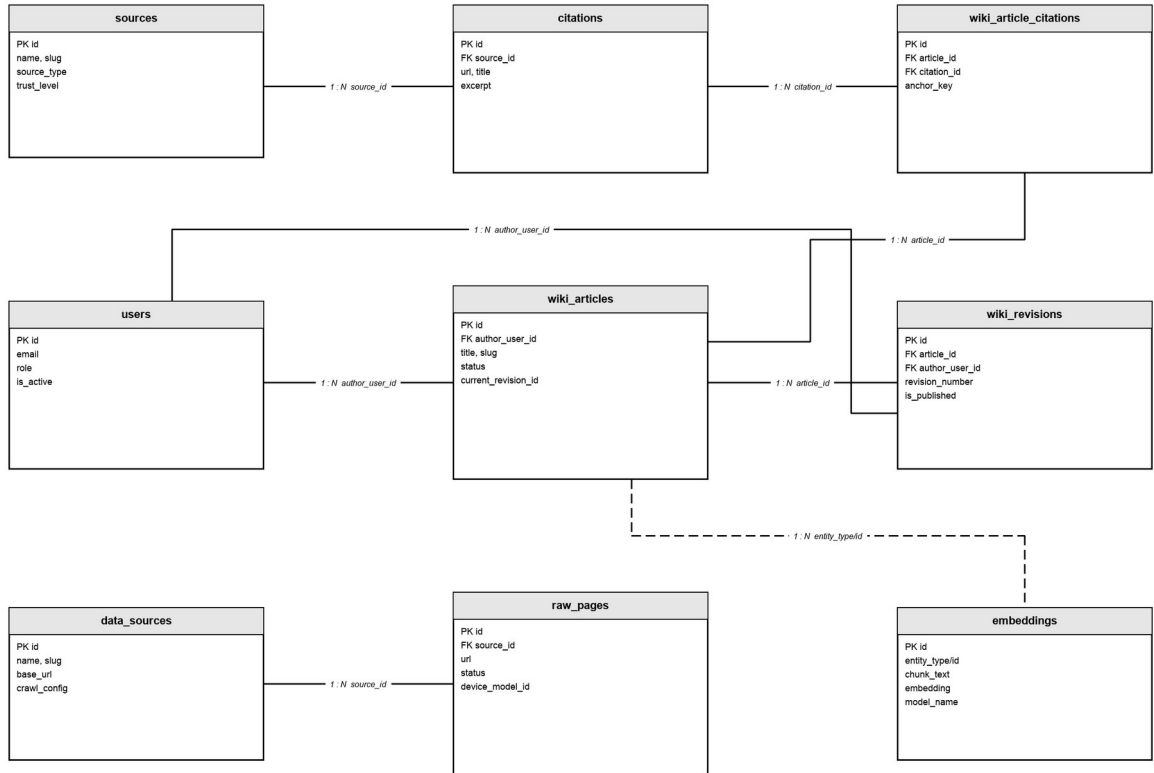
Catalog tách device_model khỏi device_variant vì tên sản phẩm và marketing content có thể dùng chung, trong khi SKU, màu, launch price và currency thuộc variant. device_model_aliases hỗ trợ search theo tên khác mà không nhân bản model; variant_price_history lưu price point theo thời gian và tham chiếu trực tiếp device_variant.



Ký hiệu 1 : N = one-to-many; nét đứt = logical reference, không phải physical foreign key.

Hình 3.9. ERD nhóm hardware và scoring

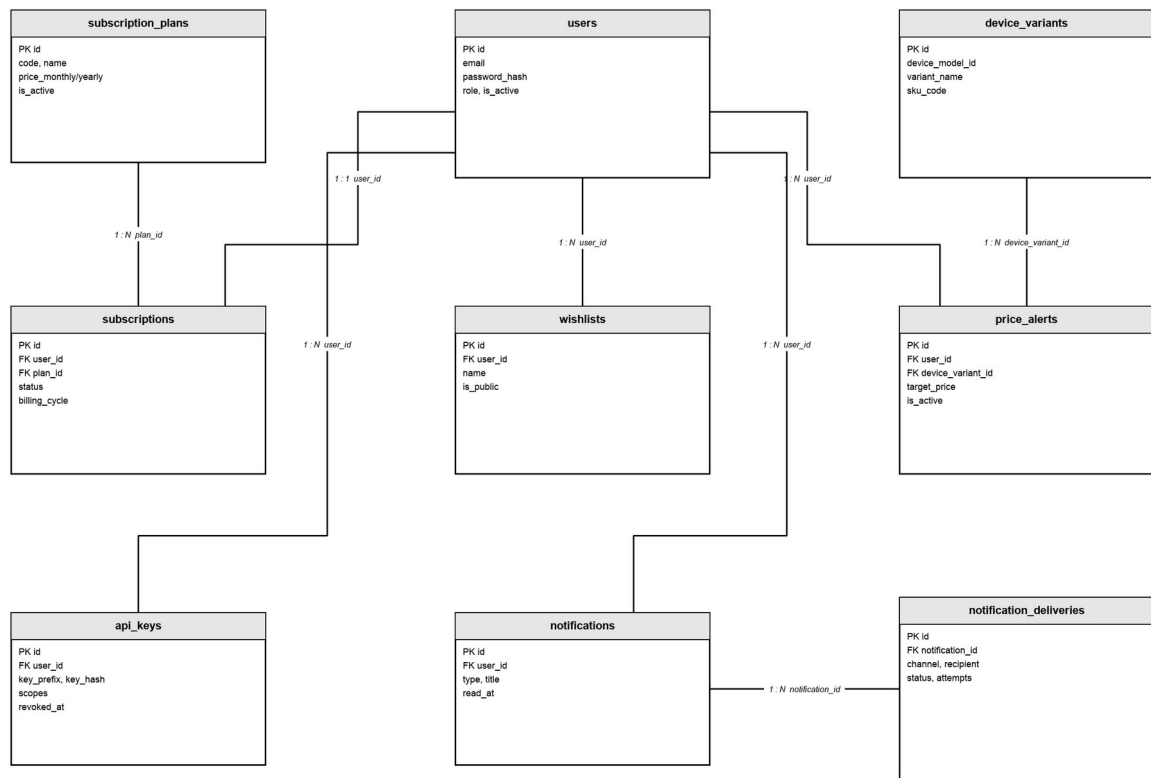
Hardware component có thể tái sử dụng giữa nhiều device_variant. Các join table giữ role hoặc position của lần gắn component. scoring_profiles tách scoring policy khỏi variant_scorecards; device_variant_benchmarks tách benchmark definition khỏi measured result để giữ traceability theo từng lần đo.



Ký hiệu 1 : N = one-to-many; nét đứt = logical reference, không phải physical foreign key.

Hình 3.10. ERD nhóm content, crawler và AI/search

wiki_revisions là immutable revision; wiki_articles giữ current_revision_id. wiki_article_citations nối article với citation. data_sources và raw_pages thuộc crawler domain. embeddings sử dụng polymorphic logical reference entity_type/entity_id; connector nét đứt nhấn mạnh đây không phải physical foreign key trong Prisma schema.



Ký hiệu 1 : N = one-to-many; nét đứt = logical reference, không phải physical foreign key.

Hình 3.11. ERD nhóm user engagement, subscription và B2B

notifications biểu diễn notification intent; notification_deliveries biểu diễn từng delivery attempt theo channel. Cách tách này cho phép retry và đo delivery status mà không tạo notification trùng. subscriptions tham chiếu subscription_plans; api_keys chỉ lưu key_hash và scopes; price_alerts tham chiếu đồng thời users và device_variants.

3.6. Thiết kế API và data contract

API sử dụng URI version v1 để cho phép thay đổi hợp đồng có kiểm soát. Resource path ưu tiên danh từ; POST dùng cho tạo hoặc action có side effect; PATCH cho cập nhật một phần; DELETE cho thu hồi/xóa theo chính sách. DTO không lộ trực tiếp cấu trúc Prisma và chỉ trả trường cần thiết theo quyền.

Bảng 3.12. Quy ước hợp đồng API

Khía cạnh	Quy ước	Lý do
Đường dẫn cơ sở	/api/v1	Tách hạ tầng và phiên bản URI

Khía cạnh	Quy ước	Lý do
Dữ liệu đầu vào	DTO + ValidationPipe; whitelist/forbid/transform	Không cho trường lạ đi vào lớp dịch vụ
Authentication	JWT/piên hoặc API key; @Public khi cần	Mặc định yêu cầu authentication, ngoại lệ được khai báo rõ
Authorization	ADMIN / EDITOR / MODERATOR	Thực hiện kiểm soát tại máy chủ
Phân trang	page/pageSize hoặc cursor có giới hạn	Tránh tải tập dữ liệu không giới hạn
Lỗi	Mã trạng thái HTTP + thông báo + request ID	Máy khách xử lý nhất quán và hỗ trợ truy vết
Tính idempotent	Khóa hoặc trạng thái duy nhất cho webhook/tác vụ/thao tác	Chống thực hiện lặp khi retry
Quan sát	Độ trễ, trạng thái, mẫu tuyến; không ghi bí mật vào nhật ký	Đo vận hành mà vẫn bảo vệ dữ liệu

3.7. Thiết kế an toàn và kiểm soát chất lượng

Mô hình đe dọa tập trung vào bốn nhóm tài sản: tài khoản và session; dữ liệu danh mục có nguồn; API key và dữ liệu sử dụng; context và câu trả lời AI. Biên tin cây xuất hiện giữa trình duyệt với API, API với dịch vụ ngoài, worker với nguồn dữ liệu và webhook với hệ thống thanh toán. Tại mỗi biên cần authentication nguồn, giới hạn dữ liệu đầu vào, quy định timeout và ghi nhật ký sự kiện.

- Quyền sở hữu: người dùng chỉ được sửa danh sách yêu thích và cảnh báo của mình; biên tập viên không mặc nhiên có quyền quản trị tài khoản.
- Tính toàn vẹn: sử dụng database transaction khi xuất bản nhiều bảng; phiên bản đã xuất bản là bất biến; webhook bảo đảm idempotency.
- Tính bí mật: mật khẩu, API key và mã authentication chỉ được lưu dưới dạng băm hoặc bí mật; nhật ký phải loại bỏ dữ liệu nhạy cảm.
- Tính khả dụng: rate limit, kích thước trang và tải tin; worker retry theo khoảng lùi và readiness check của phụ thuộc.
- Chất lượng dữ liệu: lưu content hash, citation, trạng thái kiểm duyệt, audit log và thống kê trường còn thiếu.

- Chất lượng AI: chỉ dùng context đã xuất bản, kiểm tra citation, quy định timeout, có fallback và bộ câu hỏi benchmark.

Kiểm soát chất lượng được đặt trước và sau hiện thực: lint/typecheck/schema validation ngăn lỗi cấu trúc; unit test kiểm tra quy tắc; readiness kiểm tra kết nối; ảnh giao diện xác nhận luồng; còn đo tải, E2E và test security động là các lớp cần bổ sung trước sản xuất.

3.8. Kết luận chương

Chương 3 đã chuyển các yêu cầu thành tám use case, ba sequence diagram, hai quy trình nền và bốn ERD theo miền. Thiết kế nhấn mạnh ranh giới quyền, dữ liệu đã xuất bản, idempotency, citation và khả năng truy vết. Chương 4 trình bày quá trình hiện thực các thành phần này trong mã nguồn, giao diện và kết quả đánh giá.

CHƯƠNG 4. PHÁT TRIỂN VÀ KẾT QUẢ

4.1. Môi trường và quy trình phát triển

Dự án yêu cầu Node.js từ phiên bản 22.11, pnpm 9.15 cùng PostgreSQL và Redis. Quy trình phát triển gồm cài đặt các thư viện phụ thuộc tại thư mục gốc, cấu hình biến môi trường từ tệp mẫu, khởi động hạ tầng, thực hiện migration và tạo seed data khi cần, sau đó chạy giao diện web, API và worker bằng Turborepo. Mỗi ứng dụng vẫn có lệnh riêng để hỗ trợ gỡ lỗi độc lập.

Mã nguồn TypeScript được lint và kiểu dữ liệu; bộ test đơn vị của API được thực thi bằng Jest; công cụ dòng lệnh Prisma kiểm tra schema dữ liệu. Swagger hỗ trợ tra cứu API trong môi trường phát triển. Các điểm kiểm tra trạng thái tổng hợp, liveness, readiness và metrics được tách riêng để phân biệt tiến trình đang chạy với khả năng tiếp nhận lưu lượng.

Bảng 4.1. Môi trường phần mềm

Thành phần	Yêu cầu / cấu hình	Mục đích đánh giá
Node.js	$\geq 22.11.0$	Môi trường thực thi thống nhất cho Next.js, NestJS và chuỗi công cụ
pnpm	9.15.0	Cài đặt repository và tệp khóa có thể tái lập
PostgreSQL	16 + pgvector/pg_trgm/unaccent	Schema quan hệ, tìm kiếm và embedding vector
Redis	Kết nối bằng biến môi trường	Phiên, cache và điều phối
Web	127.0.0.1:3000 khi kiểm tra cục bộ	22 web route và PWA
API	127.0.0.1:4000, cơ sở /api/v1	186 điểm cuối, kiểm tra trạng thái và Swagger trong môi trường phát triển
Test	Jest trong phạm vi API	36 bộ / 189 ca test trong lần đánh giá
Prisma	kiểm tra/tạo mã/migration	Kiểm tra schema và Prisma Client

4.2. Hiện thực các module chính

Kết quả khảo sát mã nguồn cho thấy hệ thống không dồn toàn bộ chức năng vào một API tổng quát. Các module được tổ chức theo miền nghiệp vụ và sử dụng lớp dịch vụ để phối hợp Prisma, Redis, chức năng tìm kiếm hoặc dịch vụ bên ngoài. Bảng 4.2 tóm tắt các chức năng đã được cài đặt; Phụ lục A liệt kê toàn bộ controller API.

Bảng 4.2. Module hiện thực chính

Module	Chức năng	Kỹ thuật nổi bật
Catalog/Hardware	Model, variant, linh kiện, media, alias, score	Prisma relation, slug, filter, DTO
Search/Compare	Tìm text/facet và dựng ma trận so sánh	pg_trgm/unaccent; Meilisearch tùy chọn; batch hydrate
Auth/User	Register/login/refresh/me/logout và hồ sơ	JWT, password hash, Redis session, RBAC
AI	Ask, chat, recommendation, context/citation	Retrieval, embedding, cache, model adapter
Wiki/Moderation	Article, revision, citation, comment và review	Versioning, diff, trạng thái và role
Wishlist/Alerts	Theo dõi variant, ngưỡng giá và notification	Ownership, worker, idempotency, delivery
Affiliate/Price	Đối tác, link, click, lịch sử và insight	Sync, inspect, audit và attribution
Billing/B2B	Subscription, webhook, API key và usage	Stripe, signature, hash key, scope/quota
Admin/Observability	Dashboard, audit, health và metrics	Guard, request ID, readiness và log

Một nguyên tắc hiện thực đáng chú ý là public endpoint và role không được suy ra từ tên đường dẫn. Decorator/guard trên server xác định quyền; đối với các thao tác chỉnh sửa hoặc quản trị, service vẫn phải kiểm tra ownership/trạng thái nghiệp vụ. Điều này tránh tình huống một endpoint có URL công khai nhưng vô tình cho phép hành động đặc quyền.

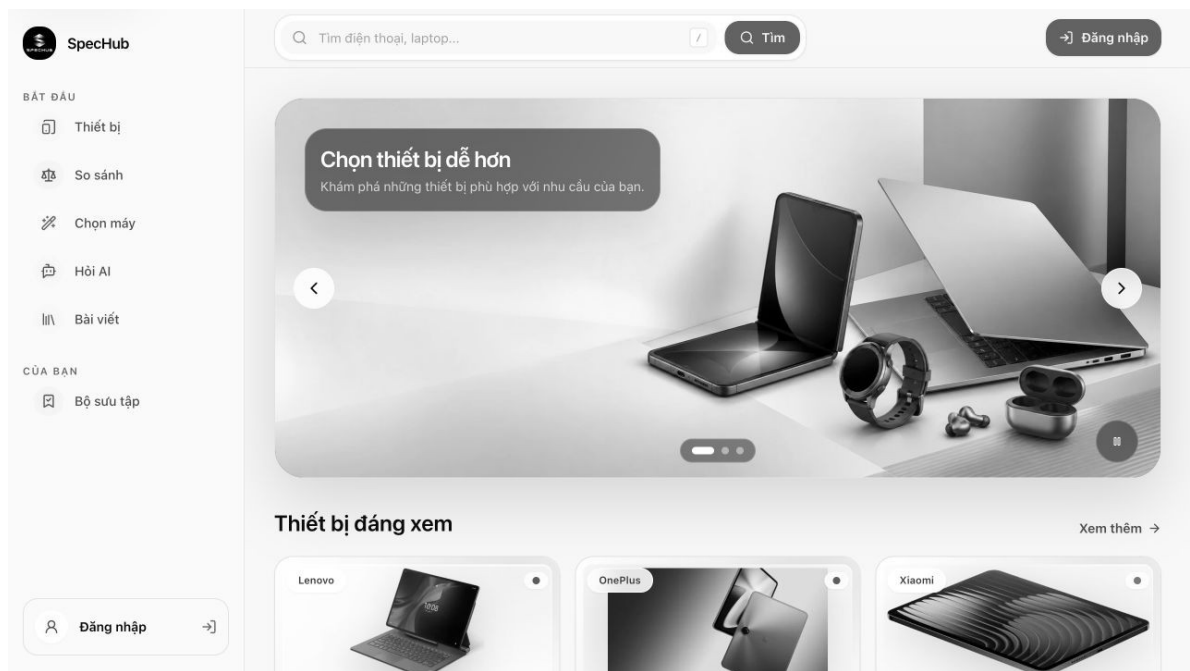
4.3. Kết quả giao diện

Giao diện được đánh giá ở độ phân giải 1440×900 trên hệ thống vận hành cục bộ với dữ liệu hiện có. Các hình dưới đây được chụp trực tiếp từ ứng dụng và minh họa những chức năng chính. Tại thời điểm ngày 05/08/2026, trang danh mục hiển thị 282 mẫu thiết

bị, 39 hãng và 8 loại thiết bị; các số liệu này phản ánh trạng thái dữ liệu tại thời điểm đánh giá.

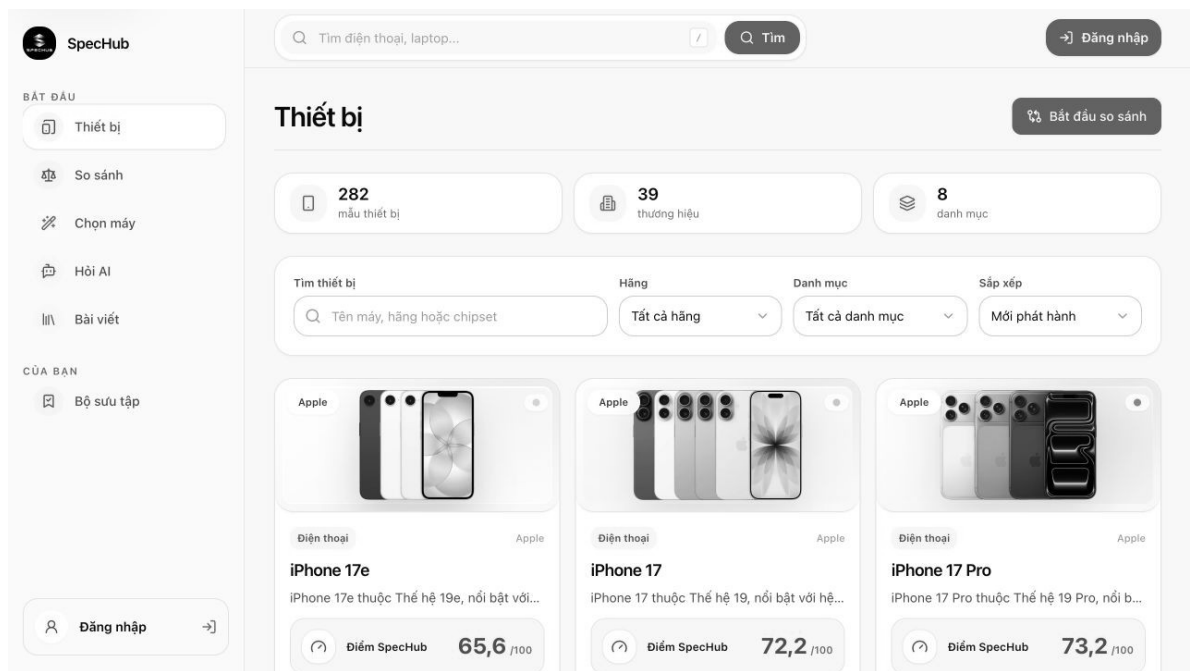
Bảng 4.3. Bản đồ route giao diện

Tuyến	Vai trò	Tuyến	Vai trò
/	Trang chủ/điều hướng khám phá	/login	Đăng nhập
/admin	Catalog Studio và quản trị	/notifications	Thông báo
/ai	Trợ lý AI	/offline	Trạng thái PWA offline
/alerts	Cảnh báo giá	/recommend	Khuyến nghị
/api-access	API key và usage	/register	Đăng ký
/billing	Gói thuê bao	/search	Tìm kiếm
/compare	So sánh	/wiki	Danh mục Wiki
/dashboard	Bảng điều khiển cá nhân	/wiki/[slug]	Chi tiết Wiki
/devices	Danh mục thiết bị	/wiki/[slug]/edit	Sửa Wiki
/devices/[slug]	Chi tiết model	/wiki/new	Tạo Wiki
/hardware/[kind]/[slug]	Chi tiết phần cứng	/wishlist	Danh sách yêu thích



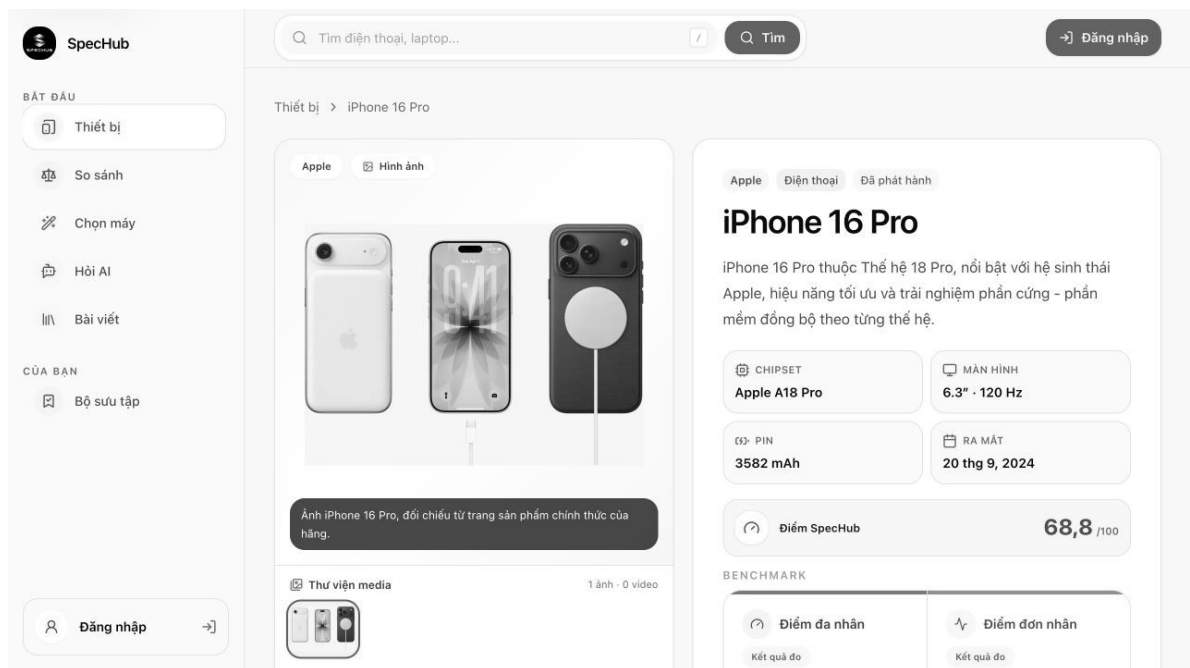
Hình 4.1. Trang chủ SpecHub

Trang chủ ưu tiên hai hành động: bắt đầu tìm kiếm và đi vào các thiết bị nổi bật. Hero dẫn dắt giá trị cốt lõi theo ngôn ngữ người dùng; thanh điều hướng giữ các đích Search, Devices, Compare, Recommend, AI và Wiki ở mức dễ thấy.



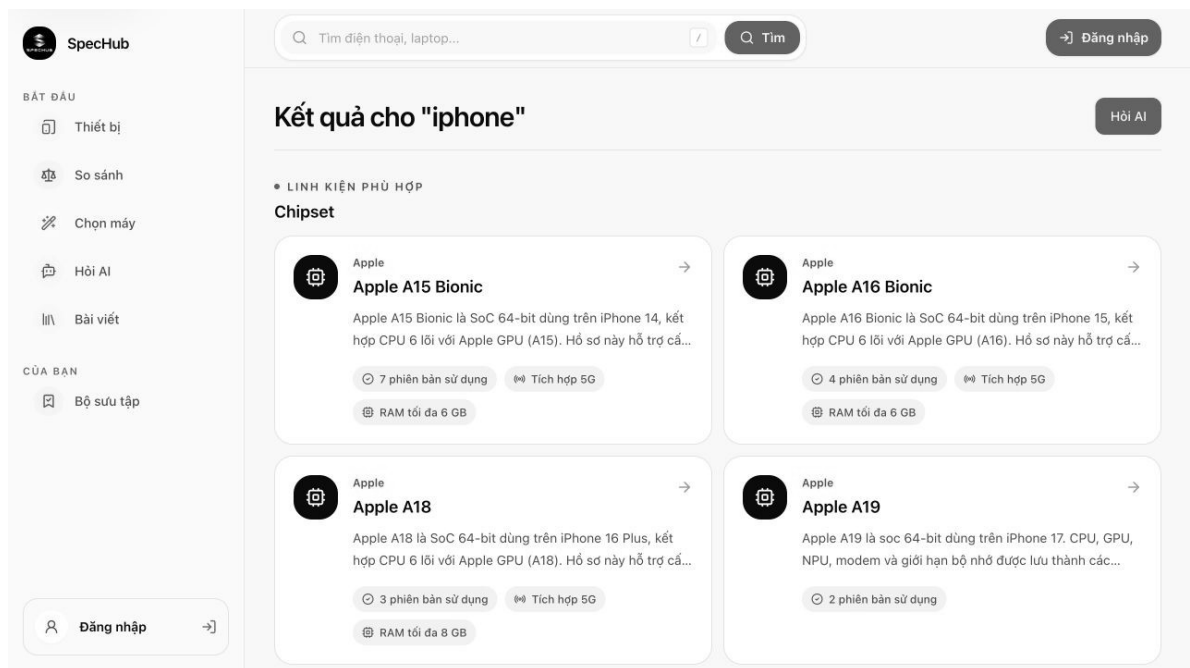
Hình 4.2. Trang danh mục thiết bị

Trang danh mục kết hợp số liệu tổng quan, lọc và danh sách thẻ. Phân trang/lọc ở server giúp URL có thể chia sẻ; trạng thái rỗng và lỗi cần được phân biệt để người dùng biết nên đổi bộ lọc hay retry.



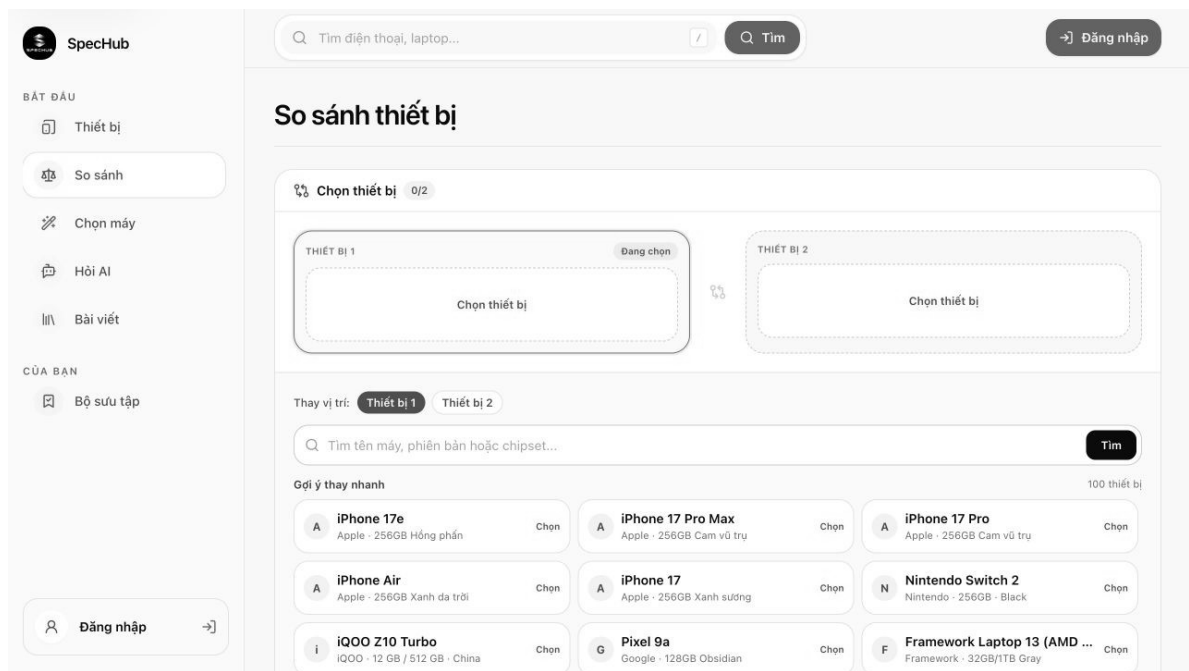
Hình 4.3. Trang chi tiết thiết bị

Trang chi tiết gom thông tin model, media, variant và phân cứng vào một context. Việc hiển thị điểm cần đi kèm profile/trọng số hoặc diễn giải; nguồn nên đặt gần thuộc tính thay vì chỉ ở cuối trang để tăng khả năng kiểm chứng.



Hình 4.4. Kết quả tìm kiếm thiết bị và phần cứng

Kết quả tìm kiếm không giới hạn ở model mà có thể đưa ra thực thể phần cứng, giúp người nghiên cứu đi từ tên thương mại tới chipset hoặc linh kiện. Nhãn loại kết quả và slug rõ ràng giúp tránh nhầm lẫn giữa model, variant và component.

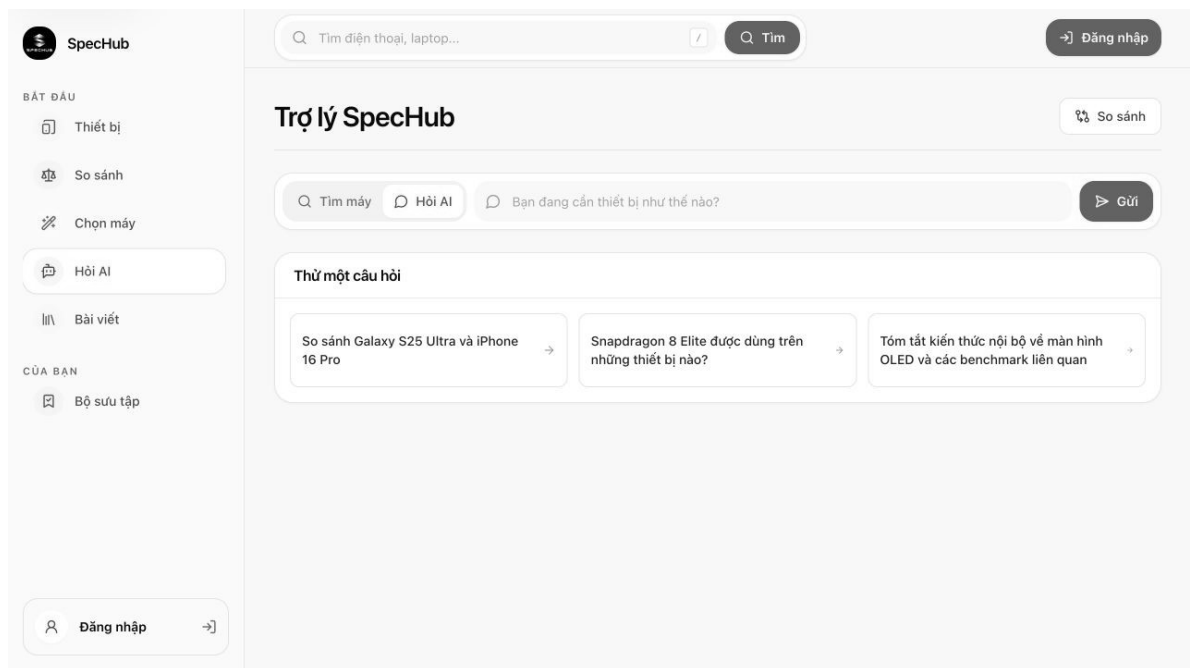


Hình 4.5. Giao diện chọn thiết bị để so sánh

Giao diện so sánh bắt đầu bằng lựa chọn có kiểm soát thay vì hiển thị bảng trống. Sau khi đủ model, ma trận cần ghim cột tên, nhóm thuộc tính, làm nổi khác biệt và không điền giá trị suy đoán cho trường thiếu.

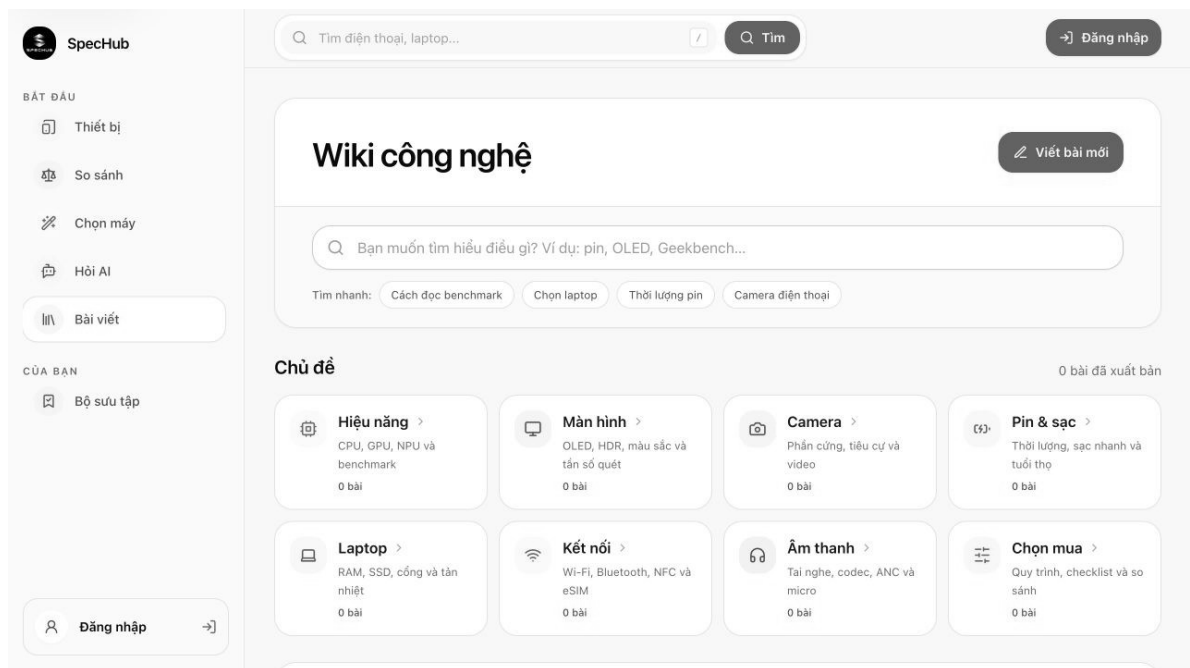
Hình 4.6. Quy trình khuyến nghị theo nhu cầu

Khuyến nghị dùng biểu mẫu ba bước để chuyển mục tiêu trừu tượng thành tiêu chí có cấu trúc trước khi gọi service. Cách này giảm độ mơ hồ, giúp giải thích vì sao một thiết bị được chọn và tạo fallback khi dịch vụ LLM không sẵn sàng.



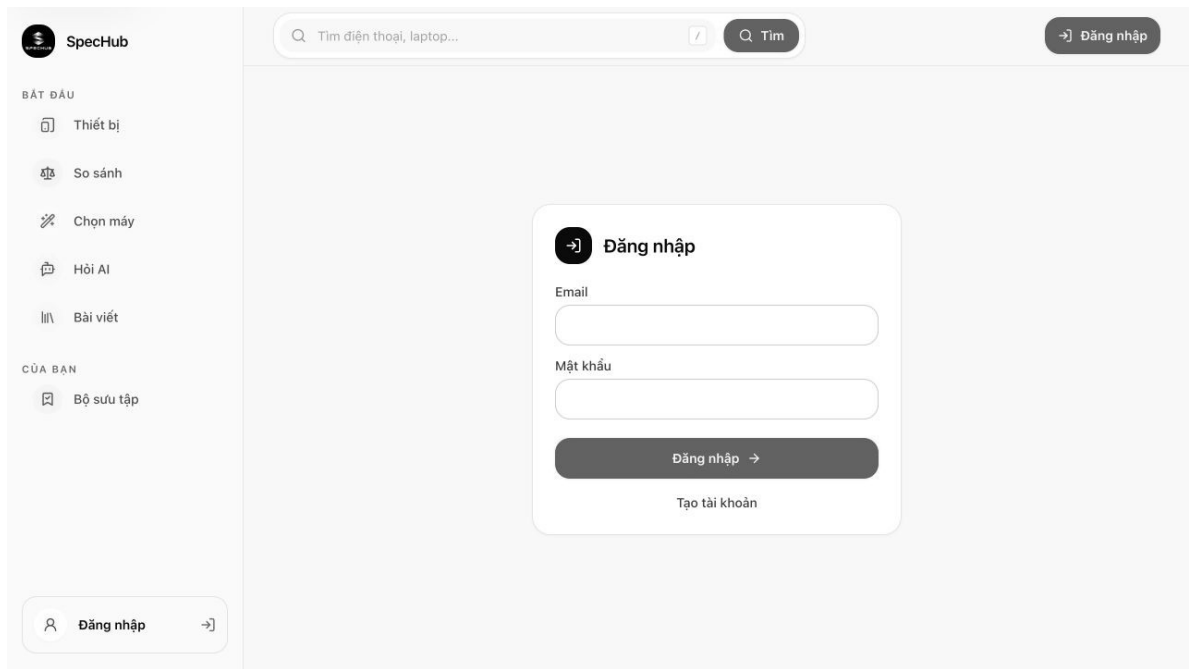
Hình 4.7. Giao diện trợ lý AI

Giao diện AI cần dành không gian cho citation, cảnh báo giới hạn và nút mở thực thể catalog. Nội dung hội thoại không được che khuất nguồn; câu trả lời nên hỗ trợ truy vấn tiếp theo nhưng không biến trạng thái chat thành nguồn sự thật cho lần retrieval khác.



Hình 4.8. Giao diện Wiki công nghệ

Snapshot Wiki đang ở trạng thái chưa có bài xuất bản trong các bộ đếm hiển thị. Điều này được giữ nguyên như bằng chứng thực tế thay vì điền dữ liệu minh họa. Luồng tạo/sửa/duyệt đã có route và mô hình; độ phong phú nội dung là công việc dữ liệu tiếp theo.



Hình 4.9. Giao diện đăng nhập

Biểu mẫu đăng nhập tối giản, nhưng kiểm soát an toàn nằm ở API: validate, rate limit, kiểm tra băm, tạo session Redis và cookie. Thông báo sai thông tin cần đủ hữu ích nhưng không tiết lộ tài khoản có tồn tại.

4.4. Kết quả API và dữ liệu

Kết quả phân tích tĩnh các controller xác định 186 API endpoint, bao gồm chức năng công khai, chức năng dành cho người dùng và chức năng quản trị. Số lượng này thể hiện phạm vi chức năng của API, nhưng không được sử dụng như tiêu chí duy nhất để đánh giá mức độ hoàn thiện. Trong mã nguồn, 45 điểm cuối được gán vai trò ADMIN, 33 điểm cuối được gán vai trò EDITOR và 3 điểm cuối được gán vai trò MODERATOR; các điểm cuối còn lại sử dụng cơ chế authentication chung, authorization sở hữu hoặc khai báo truy cập công khai.

Bảng 4.4. Thống kê endpoint API

Nhóm	Số lượng	Diễn giải
Controller	30	Nhóm endpoint theo miền
Endpoint tổng	186	Route handler được phát hiện

Nhóm	Số lượng	Diễn giải
GET	91	Đọc catalog, health, dashboard và tra cứu
POST	57	Tạo, đăng nhập, action, chat và đồng bộ
PATCH	24	Cập nhật một phần và chuyển trạng thái
DELETE	13	Xóa/thu hồi resource
PUT	1	Thay thế/cập nhật toàn phần ở trường hợp riêng
Role ADMIN	45	Endpoint có decorator ADMIN
Role EDITOR	33	Endpoint có decorator EDITOR
Role MODERATOR	3	Endpoint có decorator MODERATOR

Schema Prisma có 139 model và không khai báo enum Prisma; các trạng thái chủ yếu được biểu diễn bằng bảng tham chiếu hoặc trường chuỗi có kiểm soát ở tầng ứng dụng. Ưu điểm là dữ liệu tham chiếu có thể mở rộng; nhược điểm là cần bảo đảm validation và foreign key để tránh giá trị trạng thái tùy ý.

4.5. Test và đánh giá

Việc đánh giá được thực hiện trên phiên bản mã nguồn ngày 05/08/2026. Toàn bộ repository được lint, kiểm dữ liệu và build; bộ test đơn vị của API được thực thi bằng Jest; gói chấm điểm được test bằng Node test runner; schema Prisma được kiểm tra bằng công cụ dòng lệnh Prisma. Readiness của API được ghi nhận khi PostgreSQL và Redis đang hoạt động trước khi đóng các công phát triển. Kết quả chỉ được sử dụng trong đúng phạm vi phép thử: khi chưa thực hiện load test thì chưa kết luận về hiệu năng trong môi trường vận hành; khi chưa có test đầu cuối toàn tuyến thì kết quả test đơn vị chưa đại diện cho toàn bộ luồng giao diện.

Bảng 4.5. Bảng chứng test và kiểm chứng

Hạng mục	Kết quả	Phạm vi kết luận
Lint	Đạt; 2 tác vụ lint hoàn tất	Mã API và web không phát sinh lỗi từ cấu hình ESLint hiện hành
Type-check	Đạt; 7 tác vụ hoàn tất	API, web, worker, AI service, database và gói chấm điểm được TypeScript chấp nhận
Project build	Đạt; 5 tác vụ build hoàn tất	Next.js, NestJS, worker, AI service và gói chấm điểm build thành công
Test API	36 bộ đạt; 189 ca đạt; 0 thất bại	Quy tắc, controller và dịch vụ có test trong apps/api
Test chấm điểm	4 ca đạt; 0 thất bại	Mốc tham chiếu, dữ liệu đo và tổng trọng số hồ sơ điểm
Prisma validate	Schema hợp lệ	Cú pháp, quan hệ và cấu hình Prisma được CLI chấp nhận
Health tổng	status = ok	Tiến trình API phản hồi health
Readiness database	ok	API kết nối PostgreSQL trong môi trường kiểm tra
Readiness Redis	ok	API kết nối Redis trong môi trường kiểm tra
UI smoke	9 màn hình chính được mở và chụp	Route render với dữ liệu hiện có; không thay thế E2E assertion
Static inventory	22 web route; 30 controller; 186 endpoint; 139 model	Độ rộng mã nguồn tại commit/worktree đang phân tích

Tóm tắt kết quả đánh giá
 Lint, kiểu dữ liệu và build: đạt
 Test API: 36 bộ đạt / 189 ca đạt / 0 ca thất bại
 Test gói chấm điểm: 4 ca đạt / 0 ca thất bại
 Schema Prisma: hợp lệ
 Trạng thái API: hoạt động bình thường
 Readiness: PostgreSQL đạt; Redis đạt
 Dữ liệu giao diện: 282 mẫu thiết bị / 39 hãng / 8 loại thiết bị

4.6. Đánh giá mức độ đáp ứng

Bảng 4.6. Đánh giá mức độ đáp ứng yêu cầu

Nhóm yêu cầu	Mức đáp ứng	Kết quả / nhận xét
Catalog và chi tiết	Đạt ở mức hiện thực	Route, API, schema và giao diện chi tiết; cần tiếp tục tăng độ phủ dữ liệu

Nhóm yêu cầu	Mức đáp ứng	Kết quả / nhận xét
Tìm kiếm và so sánh	Đạt ở mức hiện thực	Route/API/sequence và UI; cần benchmark relevance và tải
Khuyến nghị/AI	Đạt kiến trúc và bề mặt	Endpoint, UI và RAG design; cần bộ đánh giá chất lượng có chuẩn
Auth/RBAC	Đạt ở mức hiện thực	JWT, refresh session Redis, guard, role và unit test; cần pentest
Wiki/moderation	Đạt mô hình và luồng	Route/schema/controller; snapshot chưa có nội dung published đáng kể
Wishlist/alert	Đạt mô hình và luồng	Route, API, worker/notification design; cần E2E theo thời gian
B2B/billing	Có nền tảng	API key, usage, subscription, Stripe/webhook; cần thử nghiệm đối tác thật
Vận hành	Đạt mức cơ bản	Kiểm tra trạng thái, readiness, metrics và request ID; chưa có SLO và kết quả đo tải trong môi trường vận hành

4.7. Hạn chế

Bảng 4.7. Hạn chế và tác động

Hạn chế	Tác động	Ưu tiên xử lý
Chưa có benchmark tải trong báo cáo	Không thể công bố p95/p99 hoặc số người dùng đồng thời	Cao: k6/Artillery theo search, compare, AI và auth
Độ phủ dữ liệu thay đổi theo seed/nguồn	Một số category/Wiki có thể ít nội dung	Cao: KPI completeness và pipeline nguồn
Chưa có E2E toàn tuyến được dẫn chứng	Unit test không phát hiện mọi lỗi trình duyệt-API	Cao: Playwright cho hành trình chính
AI chưa có bộ chấm chuẩn	Khó định lượng độ đúng/citation	Cao: golden set, RAG metrics và human review
Nhiều data model	Migration và ownership phức tạp	Trung bình: data dictionary, domain owner và migration policy
Dịch vụ ngoài có chế độ tùy chọn	Hành vi khác nhau theo môi trường	Trung bình: contract test và fallback matrix
Chưa chứng minh triển khai đa vùng	Khả dụng/khôi phục thảm họa chưa được đánh giá	Sau MVP: backup restore drill, RPO/RTO và chaos test

Các hạn chế được trình bày nhằm phân biệt rõ giữa kết quả đã đạt được và mục tiêu phát triển tiếp theo. Mỗi hạn chế cần được chuyển thành phép đo, ca test hoặc tiêu chí nghiệm thu cụ thể trong các giai đoạn tiếp theo.

4.8. Kết luận chương

Chương 4 đã đối chiếu thiết kế với mã nguồn, giao diện và kết quả test. Spechub đã xây dựng được phạm vi chức năng tương đối rộng, data model chi tiết và các thành phần vận hành cơ bản. Lint, kiểu dữ liệu và build đều đạt; toàn bộ 189 ca test API và 4 ca test chấm điểm đều đạt; schema Prisma hợp lệ; PostgreSQL và Redis đáp ứng readiness check. Tuy nhiên, mức độ sẵn sàng triển khai thực tế còn phụ thuộc vào kết quả load test, test đầu cuối, đánh giá chất lượng AI, chất lượng dữ liệu và cơ chế giám sát theo mục tiêu mức dịch vụ.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đồ án đã phân tích và tài liệu hóa một nền tảng nghiên cứu thiết bị có cấu trúc từ lớp trải nghiệm đến dữ liệu. Spechub giải quyết mối liên hệ giữa model, variant, phần cứng, nội dung, nguồn, người dùng và giá bằng 139 Prisma model; cung cấp 22 route web và 186 endpoint API theo các miền catalog, search, AI, Wiki, alerts, commerce và vận hành.

Kiến trúc sử dụng Next.js/React, NestJS/Fastify, Prisma/PostgreSQL, Redis và worker trong monorepo. Các quyết định quan trọng gồm tách published khỏi draft/raw, citation khỏi nội dung, notification khỏi delivery, session khỏi token và retrieval khỏi generation. Những ranh giới này giúp hệ thống có khả năng truy vết, test và mở rộng tốt hơn.

Kết quả đánh giá cho thấy lint, kiểu dữ liệu và build đều đạt; 36 bộ với 189 ca test API và 4 ca test chấm điểm đều đạt; schema Prisma hợp lệ; PostgreSQL và Redis đáp ứng readiness check. Các giao diện chính có thể truy cập và hiển thị dữ liệu tại thời điểm đánh giá. Những kết quả này chưa thay thế cho load test, test đầu cuối, penetration test hoặc đánh giá định lượng chất lượng AI, nhưng tạo được baseline cho các giai đoạn hoàn thiện tiếp theo.

2. Hướng phát triển

1. Xây dựng bộ KPI chất lượng catalog: độ đầy đủ theo category, freshness, conflict rate, citation coverage và thời gian review.
2. Bổ sung Playwright E2E cho đăng nhập, tìm kiếm, so sánh, khuyến nghị, Wiki review và cảnh báo giá; chạy trong CI với dữ liệu seed ổn định.
3. Thiết lập load test và SLO cho search, device detail, compare, auth refresh và AI; theo dõi p50/p95/p99, error rate và saturation.
4. Tạo golden set cho AI/RAG, đo context recall, citation precision, faithfulness và chất lượng khuyến nghị; có đánh giá con người định kỳ.
5. Hoàn thiện pipeline ingestion theo từng nguồn với parser contract test, cảnh báo thay đổi DOM, ưu tiên nguồn chính thức và chính sách tuân thủ.
6. Nâng cấp an toàn bằng threat-model review, dependency scanning, secret scanning, pentest, backup/restore drill và diễn tập thu hồi khóa; đồng thời mở rộng API B2B và Catalog Studio từ phản hồi của đối tác và biên tập viên thực tế.

TÀI LIỆU THAM KHẢO

- [1] Kho mã nguồn Spechub, tệp README và tài liệu kiến trúc nội bộ của dự án, truy cập ngày 05/08/2026.
- [2] Next.js Documentation, <https://nextjs.org/docs>, truy cập ngày 05/08/2026.
- [3] React Documentation, <https://react.dev/>, truy cập ngày 05/08/2026.
- [4] NestJS Documentation, <https://docs.nestjs.com/>, truy cập ngày 05/08/2026.
- [5] Fastify Documentation, <https://fastify.dev/docs/latest/>, truy cập ngày 05/08/2026.
- [6] Prisma ORM Documentation, <https://www.prisma.io/docs/orm>, truy cập ngày 05/08/2026.
- [7] PostgreSQL 16 Documentation, <https://www.postgresql.org/docs/16/>, truy cập ngày 05/08/2026.
- [8] pgvector Documentation, <https://github.com/pgvector/pgvector>, truy cập ngày 05/08/2026.
- [9] Redis Documentation, <https://redis.io/docs/latest/>, truy cập ngày 05/08/2026.
- [10] Meilisearch Documentation, <https://www.meilisearch.com/docs>, truy cập ngày 05/08/2026.
- [11] OWASP Application Security Verification Standard, <https://owasp.org/www-project-application-security-verification-standard/>, truy cập ngày 05/08/2026.
- [12] OWASP API Security Top 10, <https://owasp.org/www-project-api-security/>, truy cập ngày 05/08/2026.
- [13] OpenAI, Retrieval and model prompting documentation, <https://platform.openai.com/docs/>, truy cập ngày 05/08/2026.
- [14] Stripe Documentation – Webhooks and subscriptions, <https://docs.stripe.com/>, truy cập ngày 05/08/2026.
- [15] W3C, Web Content Accessibility Guidelines (WCAG) 2.2, <https://www.w3.org/TR/WCAG22/>, truy cập ngày 05/08/2026.

PHỤ LỤC A. DANH MỤC API

Bảng A.1 được tổng hợp từ các controller trong mã nguồn. Trường hợp tiền tố rỗng hoặc controller không có hàm xử lý trực tiếp vẫn được giữ lại để phản ánh đúng cấu trúc module. Thông tin chi tiết về từng API endpoint có thể được tra cứu bằng Swagger trong môi trường phát triển.

Bảng A.1. Danh mục controller và endpoint

STT	Controller	Tiền tố	Số API endpoint	Phân bố phương thức
1	app	(gốc)	0	–
2	health	health	4	GET:4
3	admin-dashboard	admin/dashboard	1	GET:1
4	affiliate	affiliate	13	DELETE:1, GET:4, PATCH:2, POST:6
5	ai	ai	10	GET:2, POST:8
6	alerts	alerts	5	DELETE:1, GET:1, PATCH:1, POST:2
7	api-keys	api-keys	4	DELETE:1, GET:1, POST:2
8	auth	auth	5	GET:1, POST:4
9	b2b	b2b	3	GET:3
10	battery-units	battery-units	3	GET:3
11	camera-modules	camera-modules	3	GET:3
12	catalog-studio	admin/catalog-studio	17	GET:7, PATCH:1, POST:8, PUT:1
13	chipsets	chipsets	3	GET:3
14	citations	citations	7	GET:3, PATCH:2, POST:2
15	data-ingestion	data-ingestion	8	GET:4, PATCH:2, POST:2
16	device-categories	device-categories	7	DELETE:1, GET:4, PATCH:1, POST:1

STT	Controller	Tiền tố	Số API endpoint	Phân bố phương thức
17	device-models	device-models	7	DELETE:1, GET:4, PATCH:1, POST:1
18	device-variants	device-variants	10	DELETE:1, GET:6, PATCH:1, POST:2
19	display-units	display-units	3	GET:3
20	admin-hardware-catalog	admin/hardware	5	DELETE:1, PATCH:1, POST:3
21	hardware-catalog	hardware	11	GET:11
22	notifications	notifications	5	GET:2, PATCH:2, POST:1
23	organizations	organizations	6	DELETE:1, GET:3, PATCH:1, POST:1
24	product-families	product-families	6	DELETE:1, GET:3, PATCH:1, POST:1
25	search	search	1	GET:1
26	subscriptions	subscriptions	12	GET:5, PATCH:2, POST:5
27	webhooks	subscriptions/webhooks	1	POST:1
28	users	users	9	DELETE:1, GET:4, PATCH:4
29	wiki	wiki	9	DELETE:1, GET:4, PATCH:1, POST:3
30	wishlists	wishlists	8	DELETE:2, GET:1, PATCH:1, POST:4

Các nhóm endpoint có bề mặt lớn gồm affiliate/price, AI, catalog, Wiki/moderation, notification, admin và billing. Khi mở rộng, cần ưu tiên tính nhất quán của DTO, status code, pagination và quyền hơn là tăng số route.

PHỤ LỤC B. DANH MỤC MÔ HÌNH DỮ LIỆU

Schema vật lý có 139 model Prisma. Bảng B.1 liệt kê toàn bộ tên model theo thứ tự xuất hiện trong schema và nhóm phân tích gần đúng. Nhóm chỉ phục vụ đọc báo cáo; quan hệ thật được định nghĩa trong Prisma.

Bảng B.1. Danh sách Prisma model

STT	Tên mô hình	Nhóm phân tích
1	languages	Catalog / tham chiếu
2	translations	Catalog / tham chiếu
3	release_statuses	Catalog / tham chiếu
4	currencies	Catalog / tham chiếu
5	sources	Nội dung / AI / nguồn
6	citations	Nội dung / AI / nguồn
7	media_assets	Catalog / tham chiếu
8	entity_media	Catalog / tham chiếu
9	module_field_coverage	Catalog / tham chiếu
10	tags	Catalog / tham chiếu
11	entity_tags	Catalog / tham chiếu
12	units	Catalog / tham chiếu
13	organizations	Catalog / tham chiếu
14	organization_roles	Catalog / tham chiếu
15	organization_role_assignments	Catalog / tham chiếu
16	device_categories	Catalog / tham chiếu
17	regions	Catalog / tham chiếu
18	product_families	Catalog / tham chiếu
19	device_models	Catalog / tham chiếu
20	device_variants	Catalog / tham chiếu
21	device_model_aliases	Catalog / tham chiếu
22	device_editorial_sections	Catalog / tham chiếu
23	variant_price_history	Người dùng / thương mại

STT	Tên mô hình	Nhóm phân tích
24	variant_physical_specs	Catalog / tham chiếu
25	variant_io_specs	Catalog / tham chiếu
26	variant_thermal_specs	Catalog / tham chiếu
27	model_lineage	Catalog / tham chiếu
28	model_similarity	Catalog / tham chiếu
29	technology_families	Catalog / tham chiếu
30	architectures	Catalog / tham chiếu
31	process_nodes	Catalog / tham chiếu
32	camera_roles	Phản cứng / điểm
33	display_technologies	Phản cứng / điểm
34	battery_chemistries	Phản cứng / điểm
35	network_generations	Catalog / tham chiếu
36	chipsets	Phản cứng / điểm
37	cpus	Phản cứng / điểm
38	cpu_clusters	Phản cứng / điểm
39	gpus	Phản cứng / điểm
40	npus	Phản cứng / điểm
41	modems	Catalog / tham chiếu
42	camera_sensors	Phản cứng / điểm
43	camera_modules	Phản cứng / điểm
44	display_units	Phản cứng / điểm
45	battery_units	Phản cứng / điểm
46	memory_standards	Phản cứng / điểm
47	storage_standards	Phản cứng / điểm
48	operating_systems	Phản cứng / điểm
49	os_versions	Catalog / tham chiếu
50	os_ui_layers	Catalog / tham chiếu
51	os_ui_layer_versions	Catalog / tham chiếu
52	cellular_bands	Catalog / tham chiếu

STT	Tên mô hình	Nhóm phân tích
53	wifi_bands	Catalog / tham chiếu
54	certifications	Catalog / tham chiếu
55	cpu_capabilities	Phản cứng / điểm
56	cpu_capability_links	Phản cứng / điểm
57	gpu_apis	Phản cứng / điểm
58	gpu_api_support	Phản cứng / điểm
59	npv_precision_capabilities	Phản cứng / điểm
60	camera_features	Phản cứng / điểm
61	camera_module_feature_links	Phản cứng / điểm
62	camera_video_modes	Phản cứng / điểm
63	camera_module_video_modes	Phản cứng / điểm
64	hdr_standards	Catalog / tham chiếu
65	display_hdr_support	Phản cứng / điểm
66	color_gamuts	Catalog / tham chiếu
67	display_color_gamut_support	Phản cứng / điểm
68	charging_protocols	Catalog / tham chiếu
69	battery_charging_protocols	Phản cứng / điểm
70	connectivity_features	Catalog / tham chiếu
71	chipset_cpu_links	Phản cứng / điểm
72	chipset_gpu_links	Phản cứng / điểm
73	chipset_npv_links	Phản cứng / điểm
74	chipset_modem_links	Phản cứng / điểm
75	camera_module_sensor_links	Phản cứng / điểm
76	variant_chipsets	Phản cứng / điểm
77	variant_cpus	Phản cứng / điểm
78	variant_gpus	Phản cứng / điểm
79	variant_npus	Phản cứng / điểm
80	variant_modems	Catalog / tham chiếu
81	variant_displays	Phản cứng / điểm

STT	Tên mô hình	Nhóm phân tích
82	variant_batteries	Catalog / tham chiếu
83	variant_camera_systems	Phản cứng / điểm
84	variant_camera_modules	Phản cứng / điểm
85	variant_memory_configs	Phản cứng / điểm
86	variant_storage_configs	Phản cứng / điểm
87	variant_wifi_bands	Catalog / tham chiếu
88	variant_operating_systems	Phản cứng / điểm
89	variant_software_profiles	Catalog / tham chiếu
90	variant_connectivity_support	Catalog / tham chiếu
91	variant_module_scores	Phản cứng / điểm
92	variant_score_metric_inputs	Phản cứng / điểm
93	variant_scorecards	Phản cứng / điểm
94	variant_scorecard_modules	Phản cứng / điểm
95	scoring_profiles	Catalog / tham chiếu
96	scoring_profile_modules	Catalog / tham chiếu
97	scoring_profile_metrics	Catalog / tham chiếu
98	variant_cellular_band_support	Catalog / tham chiếu
99	variant_certifications	Catalog / tham chiếu
100	variant_region_availability	Catalog / tham chiếu
101	software_features	Catalog / tham chiếu
102	variant_software_features	Catalog / tham chiếu
103	feature_definitions	Catalog / tham chiếu
104	device_variant_features	Catalog / tham chiếu
105	benchmarks	Phản cứng / điểm
106	benchmark_runs	Phản cứng / điểm
107	device_variant_benchmarks	Phản cứng / điểm
108	chipset_benchmarks	Phản cứng / điểm
109	cpu_benchmarks	Phản cứng / điểm
110	gpu_benchmarks	Phản cứng / điểm

STT	Tên mô hình	Nhóm phân tích
111	npv_benchmarks	Phản ứng / điểm
112	catalog_drafts	Catalog / tham chiếu
113	catalog_draft_versions	Catalog / tham chiếu
114	catalog_entity_versions	Catalog / tham chiếu
115	users	Người dùng / thương mại
116	wiki_articles	Nội dung / AI / nguồn
117	wiki_revisions	Nội dung / AI / nguồn
118	wiki_article_citations	Nội dung / AI / nguồn
119	comments	Nội dung / AI / nguồn
120	embeddings	Nội dung / AI / nguồn
121	ai_query_cache	Nội dung / AI / nguồn
122	search_logs	Nội dung / AI / nguồn
123	data_sources	Nội dung / AI / nguồn
124	raw_pages	Nội dung / AI / nguồn
125	affiliate_partners	Người dùng / thương mại
126	affiliate_links	Người dùng / thương mại
127	affiliate_price_history	Người dùng / thương mại
128	affiliate_clicks	Người dùng / thương mại
129	subscription_plans	Người dùng / thương mại
130	subscriptions	Người dùng / thương mại
131	billing_audit_logs	Người dùng / thương mại
132	billing_webhook_events	Người dùng / thương mại
133	wishlists	Người dùng / thương mại
134	wishlist_items	Người dùng / thương mại
135	price_alerts	Người dùng / thương mại
136	notifications	Người dùng / thương mại
137	notification_deliveries	Người dùng / thương mại
138	api_keys	Người dùng / thương mại
139	api_key_usage	Người dùng / thương mại

PHỤ LỤC C. HƯỚNG DẪN VẬN HÀNH VÀ KIỂM THỬ

Các lệnh dưới đây mang tính quy trình; tên script cụ thể phải đối chiếu package.json tại thời điểm chạy. Không đưa bí mật hoặc giá trị production vào tài liệu. Tập .env chỉ được tạo từ mẫu và quản lý ngoài kiểm soát phiên bản.

Bảng C.1. Ma trận lệnh vận hành và mục đích

Bước	Lệnh tham chiếu	Mục đích / kết quả mong đợi
Cài đặt	<code>pnpm install --frozen-lockfile</code>	Cài đúng lockfile trong workspace
Hạ tầng	<code>pnpm docker:up</code> hoặc <code>docker compose up</code>	Khởi động PostgreSQL, Redis và dịch vụ tùy chọn
Prisma	<code>pnpm --filter database prisma validate</code>	Schema hợp lệ trước generate/migrate
Generate	<code>pnpm --filter database prisma generate</code>	Tạo Prisma client đúng schema
Migration	Prisma migrate theo môi trường	Áp migration có review; không dùng reset trên dữ liệu cần giữ
Seed	Script seed của packages/database	Tạo dữ liệu phát triển có thể tái lập
Dev	<code>pnpm dev</code>	Chạy pipeline web/API/worker theo cấu hình
Test API	<code>pnpm --filter api test</code>	Chạy Jest và báo suite/test thất bại
Typecheck	<code>pnpm typecheck</code>	Kiểm tra kiểu toàn workspace
Lint	<code>pnpm lint</code>	Kiểm tra quy ước và lỗi tĩnh
Health	<code>GET /api/v1/health/ready</code>	DB/Redis phải báo ready trước nhận lưu lượng
Backup	Quy trình <code>pg_dump/restore</code> đã phê duyệt	Kiểm tra resilience, không chỉ tạo bản sao

C.1. Checklist trước khi triển khai

- Tất cả migration đã review, backup có thể phục hồi và biến môi trường bắt buộc đã được kiểm tra.
- Test, typecheck, lint và build đều đạt trên cùng commit; image/container có version bất biến.

- CORS, cookie, JWT secret, webhook secret, API key hashing và rate limit đúng môi trường.
- Health/readiness/metrics, log redaction, request ID, dashboard và cảnh báo đã hoạt động.
- Index tìm kiếm và embedding vector được quản lý phiên bản; worker bảo đảm idempotency; cơ chế retry và queue lỗi được giám sát.
- Có kế hoạch rollback, người chịu trách nhiệm và tiêu chí dừng triển khai.

PHỤ LỤC D. HƯỚNG DẪN SỬ DỤNG TÓM TẮT

Hướng dẫn này mô tả hành trình ở mức người dùng và không yêu cầu hiểu cấu trúc kỹ thuật.

Bảng D.1. Tác vụ người dùng thường gặp

Tác vụ	Cách thực hiện	Lưu ý
Tìm thiết bị	Mở Search/Devices, nhập từ khóa, chọn loại/hãng và bộ lọc	Bỏ bớt filter nếu không có kết quả; kiểm tra alias
Xem chi tiết	Chọn thẻ model để xem variant, phần cứng, điểm và nguồn	Phân biệt model với variant; trường thiếu không phải giá trị 0
So sánh	Thêm từ hai thiết bị vào Compare	Ưu tiên cùng category; mở nguồn khi giá trị mâu thuẫn
Nhận khuyến nghị	Hoàn thành các bước nhu cầu/ngân sách/ưu tiên	Kết quả là hỗ trợ quyết định, không thay thế kiểm tra thực tế
Hỏi AI	Đặt câu hỏi cụ thể và xem citation	Không coi câu không có nguồn là dữ kiện đã xác minh
Wishlist/cảnh báo	Đăng nhập, lưu variant và đặt target price	Kiểm tra tiền tệ, kênh nhận và tùy chọn thông báo
Đóng góp Wiki	Tạo/sửa revision, thêm nguồn và gửi duyệt	Bản sửa không xuất bản ngay; có thể được yêu cầu bổ sung
Dùng API	Tạo API key theo gói, lưu khóa an toàn và gọi endpoint trong scope	Khóa chỉ hiển thị rõ một lần; theo dõi quota/usage

D.1. Xử lý sự cố cơ bản

- Nếu trang báo offline: kiểm tra kết nối, thử tải lại; dữ liệu cache có thể không phải giá mới nhất.
- Nếu đăng nhập hết hạn: thực hiện đăng nhập lại; không gửi token hoặc snapshot token cho người khác.
- Nếu kết quả sai: mở citation, ghi URL, thuộc tính và biến thể liên quan, sau đó gửi phản hồi kèm nguồn kiểm chứng.
- Nếu cảnh báo không đến: kiểm tra alert active, ngưỡng, kênh nhận, quiet hours và trạng thái notification.
- Nếu API trả 429: đọc header quota/retry, giảm tần suất và không tạo thêm key để né hạn mức.